

FIAP · MBA AI ENGINEERING & MULTI-AGENTS
CLOUD & COGNITIVE ENVIRONMENTS

Serverless & containers

Aula 03 — API: o endpoint que os agentes conhecem como **“tool”**.
Function, Managed Identity e o mesmo código num container.

Como vamos passar as próximas 4 horas

00:00 — 01:10

Bloco 1 — Serverless

Event-driven, pay-per-call, triggers, quando NÃO usar + lab Function HTTP via Terraform

01:10 — 02:20

Bloco 2 — Triggers & Managed Identity

Cold start, a vacina da aula: MI vs API keys + lab Function lê o Blob sem credencial

02:20 — 02:30

Break

10 minutos de pausa

02:30 — 03:35

Bloco 3 — Containers

Docker, ACR, ACI + lab: o mesmo código Python num container

03:35 — 04:00

Bloco 4 — Matriz de decisão & wrap-up

Container Apps, AKS, a matriz Function/ACI/Apps/AKS, destroy e a API como tool dos agentes

Um agente sem ferramentas é só um **chatbot genérico**.

Na Aula 2 os dados ganharam onde viver. Hoje a gente constrói a primeira **tool** de um agente — uma API de catálogo — e a constrói **duas vezes** : serverless e container. Essa decisão define custo, escala e capacidade do agente.



Onde vão viver os dados dos agentes ?

Storage e bancos são decisões arquiteturais que definem o que o agente consegue ou não fazer.

01

Serverless

Event-driven, pay-per-execution, auto-scale.

Os trade-offs vs VM e container e a primeira Function provisionada via Terraform.



```
def consultar_estoque(produto_id: str, armazem_id: str) -> dict:
    """
    Consulta a quantidade disponível de um produto no estoque.
    Tool conceitual: em produção, faria uma chamada a um banco
    de dados, ERP ou API REST de inventário.
    """

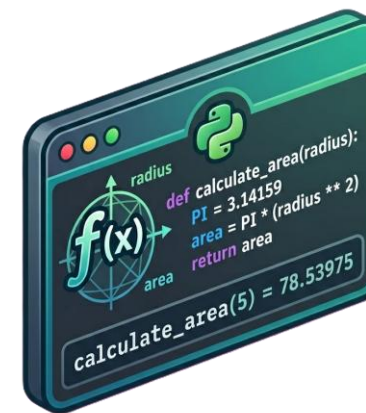
    # 1 – Valida parâmetros de Entrada
    # ...

    # 2 – Conecta no banco de dados
    # ...

    # 3 – Consulta produto
    # ...

    # 4 – Monta resposta à chamada
    retorno_estoque = { "produto_id": produto_id,
                        "nome": "Café Especial 500g",
                        "quantidade_disponivel": 42,
                        "em_estoque": True,
                        "armazem": armazem_id
                      }

    return retorno_estoque
```



Você identificou que seu agente precisará obter constantemente o estoque de produtos.

Ou seja, essa ação faz parte das funcionalidades comuns do teu agente.

Esta é uma ação **determinística**, ou seja, sempre deve ser feita da mesma forma.

Para situações como essa, não há dúvida: teu agente necessita de uma “**tool**”.

Podemos dizer então que funções são um dos tipos de ferramentas (tools) que um agente tem à disposição.



Precisamos de um lugar para hospedar suas funções.

```
def consultar_estoque(produto_id: str, armazem_id: str) -> dict:
```

```
"""
```

Consulta a quantidade disponível de um produto no estoque.
Tool conceitual: em produção, faria uma chamada a um banco de dados, ERP ou API REST de inventário.

```
"""
```

```
# 1 – Valida parâmetros de Entrada
```

```
# ...
```

```
# 2 – Conecta no banco de dados
```

```
# ...
```

```
# 3 – Consulta produto
```

```
# ...
```

```
# 4 – Monta resposta à chamada
```

```
retorno_estoque = { "produto_id": produto_id,
                    "nome": "Café Especial 500g",
                    "quantidade_disponivel": 42,
                    "em_estoque": True,
                    "armazem": armazem_id
                    }
```

```
return retorno_estoque
```



Podemos pensar em hospedar nossa função em um servidor.

Função Python + Máquina



Ou nossas funções podem ser hospedadas na nuvem, de modo “Serverless” (Sem servidor).

Apenas a Função Python



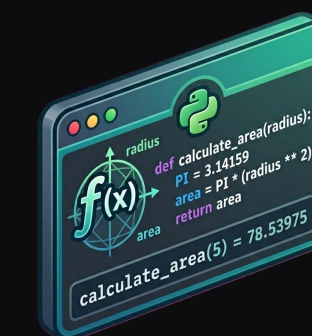
· REFORÇO ·

Serverless, ainda roda em **servidores**.

"Sem servidor" é marketing.

Tem rack, disco, refrigeração — só não é problema seu.

Você manda código; o Provedor decide onde rodar,
quantas cópias subir e quando desligar.



O Servidor existe, você é quem não gerencia

Algumas características que podemos colocar em funções Serverless

EVENT-DRIVEN

Algo acontece → roda

Uma chamada, uma mensagem, um arquivo novo dispara a função.

Sem processo ligado esperando o tempo todo.

PAY-PER-EXECUTION

Paga por chamada

~ \$0,0000002 por execução + GB-segundo de memória.

Ocioso custa **zero** — não há máquina ligada.

AUTO-SCALE

0 → centenas

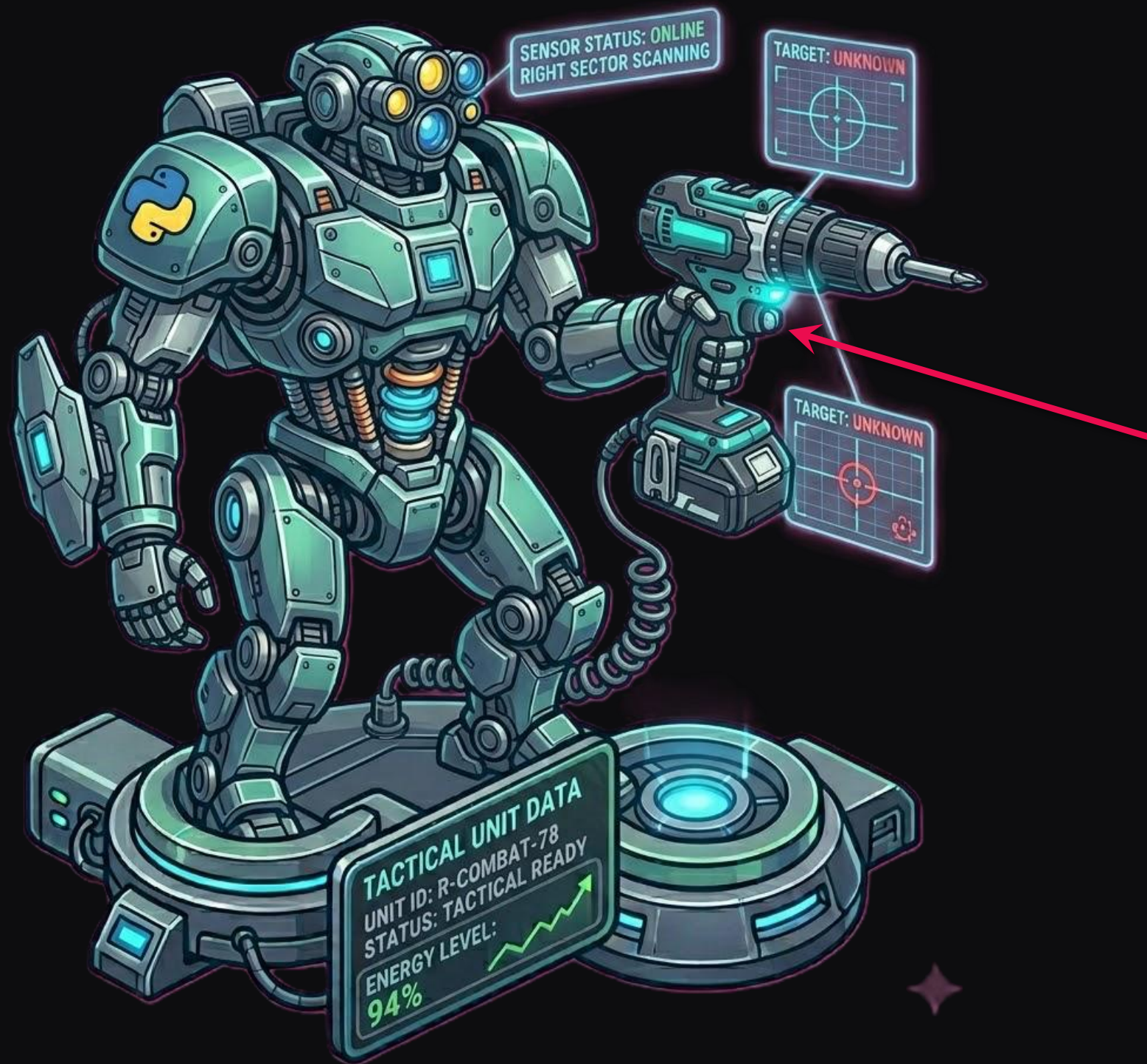
Escala sozinho conforme o tráfego e volta a zero.

Você nunca dimensiona servidor.

Free grant: 1M execuções/mês + 400.000 GB-s . Cobre ~90% das tools de agentes — uma startup inteira roda no free tier.

Gatilhos de funções

o que faz a função rodar



Para que o agente utilize sua ferramenta, essa ferramenta precisa de um gatilho que o agente saiba usar!

Os gatilhos mais comuns para aplicações ativar funções é por API HTTP (REST e MCP).

Processos Cliente



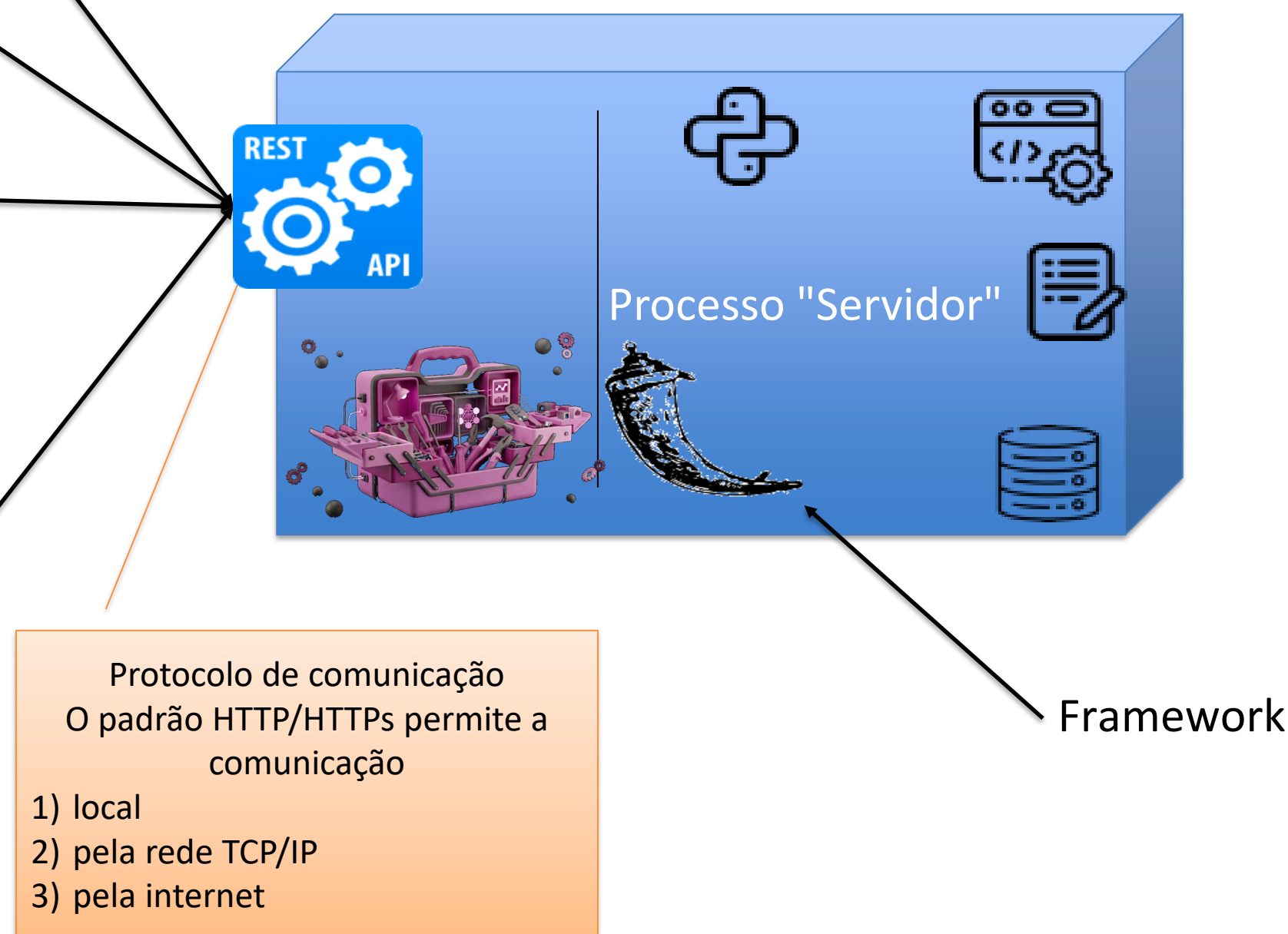
- O gatilho mais comum de integrar sistemas é através de API REST

- (mais a frente vocês verão uma variação chamada MCP, específica para agentes)

API = Application Programming Interface

REST = Representational State Transfer

Cada requisição feita à API deve conter todas as informações necessárias para ser processada.
O servidor não mantém informações de estado entre as requisições.



API e MCP não competem

O PRINCÍPIO

API é contrato entre programas. MCP é contrato entre um modelo e ferramentas. Não são camadas concorrentes: um servidor MCP quase sempre chama uma API por baixo.

MESMA CHAMADA, OUTRO CONTRATO

Quem decide a chamada

API o código que você escreveu, em tempo de build

MCP o modelo, em runtime, lendo a descrição

Como se descobre

API você lê a doc e escreve o cliente

MCP o servidor anuncia tools e schemas ao conectar

Para quem o contrato é escrito

API OpenAPI — para humano e gerador de código

MCP JSON Schema + texto — para o modelo ler

Quando a interface muda

API quebra o cliente; alguém recompila

MCP o host relê a lista na próxima sessão

O PROBLEMA QUE ELE RESOLVE

Sem protocolo comum: cada host precisa de um cliente para cada serviço.

N hosts × **M** serviços = **N×M** integrações para manter

Com MCP: **N + M**. Um servidor serve qualquer host que fale o protocolo — e o mesmo host consome qualquer servidor.

UM SERVIDOR EXPÕE TRÊS COISAS

- **tools** ações que o modelo pode executar — é o que mais se parece com endpoint
- **resources** dados que ele pode ler, endereçados por URI
- **prompts** templates que o usuário invoca, não o modelo

Atenção: a descrição da tool não é documentação, é prompt. Ela é o que o modelo lê para decidir se chama. Servidor MCP se projeta escrevendo, não só expondo.

A consequência que importa: com API, quais chamadas existem é decidido em tempo de build. Com MCP, em runtime, pelo modelo — a autorização sai do projeto e vai para a execução. Escopo mínimo e identidade passam a importar mais, não menos.

Outros gatilhos comuns, um para cada caso

HTTP

Chamada REST/MCP

Chamada por protocolo HTTP sobre TCP.

Exemplo:

O agente chama a tool pela rede/internet.

É o que usamos hoje.

TIMER

Cron

Ocorre em momento pré-determinado.

Exemplo:

Rotina agendada para limpar carrinho expirado de madrugada.

QUEUE

Fila

Mensagem nova dispara função.

Exemplo: processamento assíncrono de pedidos.

BLOB

Arquivo novo

Upload no Storage dispara a função.

Exemplo:

processar imagem de produto.

EVENT GRID

Evento Azure

Outros eventos da nuvem.

Exemplo:

VM criada, blob deletado — reage a eventos da própria nuvem.

Alguns modos de rodar o mesmo código

MODELO	VOCÊ GERENCIA	CUSTO	COLD START	QUANDO USAR
VM	SO, patches, scaling	\$/hora 24×7	Não	App legado, controle total
Function	Só o código	\$/execução	Sim (1–5s)	APIs intermitentes, eventos

Uma Terceira opção de execução são os contêineres, que abordaremos no terceiro bloco.

APIs de agente são intermitentes por natureza — chamadas sob demanda, não 24×7.
Casam com Function.

LAB 1



Function HTTP via Terraform + deploy

Antes de continuar, certifique-se de ter à disposição:

- Conta Azure já logada
- Cloud Shell funcionando
- Uma boa xícara de café ☕

1 – Preparação – cria os componentes requeridos antes do lab

Clonar o repositório (apenas na primeira vez)

```
git clone https://github.com/isaiasbritto/aie-cloud.git  
cd aie-cloud/aulas/03-serverless-containers/lab/terraform
```

Abrir o editor VS Code na pasta do lab

code .

Inicializar providers (download de bibliotecas)

terraform init

Ver o que será criado

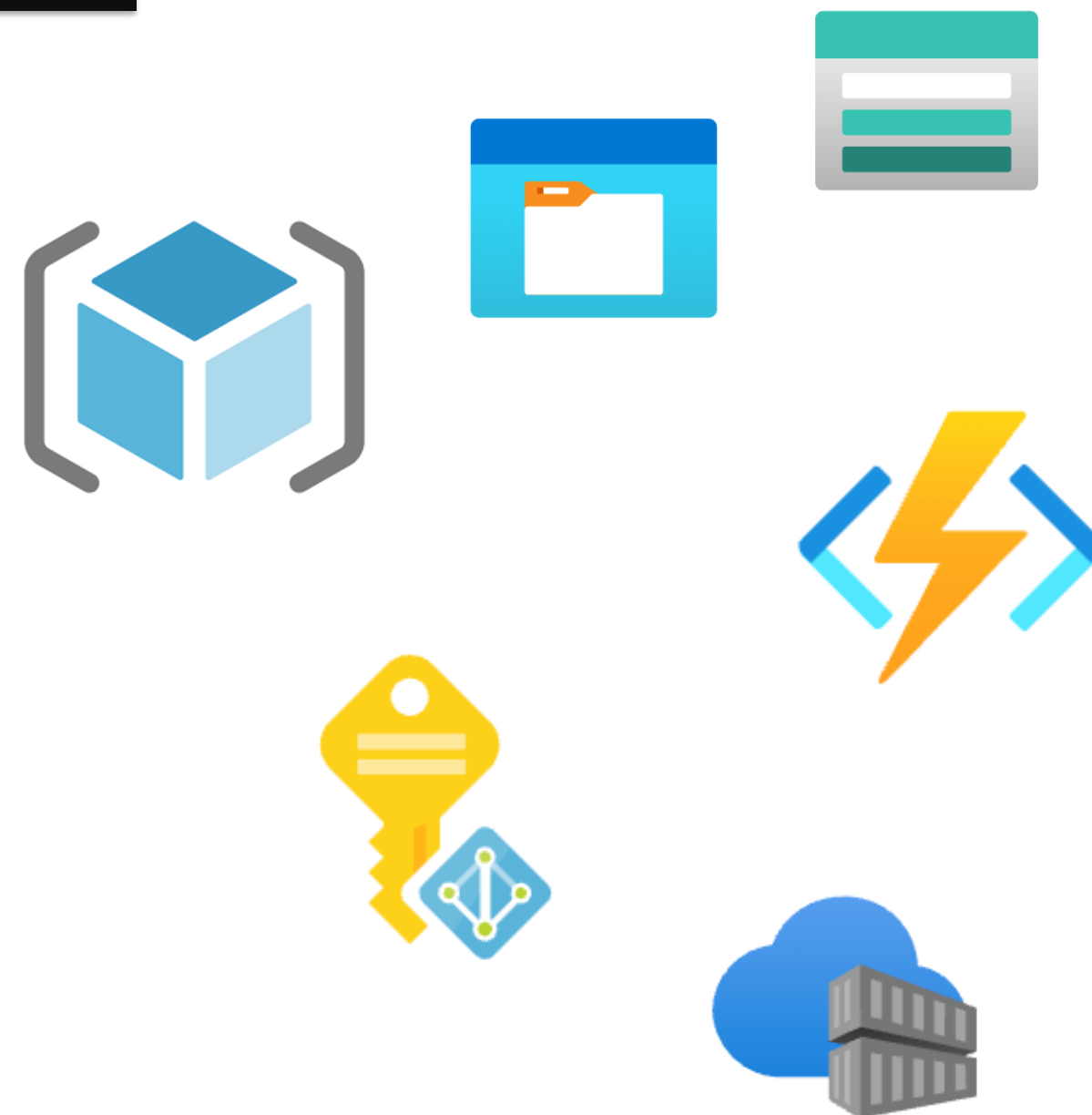
terraform plan

Aplicar

terraform apply -auto-approve

aci_enabled=false (default) → ACI não é criado

Caso o repo tenha sido clonado, apague antes com
rm -rf aie-cloud



Aproximadamente ~5 a ~8 minutos (tempo para explorar a pasta terraform)

Lembrete da regra de ouro:

Ao final, devemos encerrar tudo com **Exclusão do RG** ou com `terraform destroy`

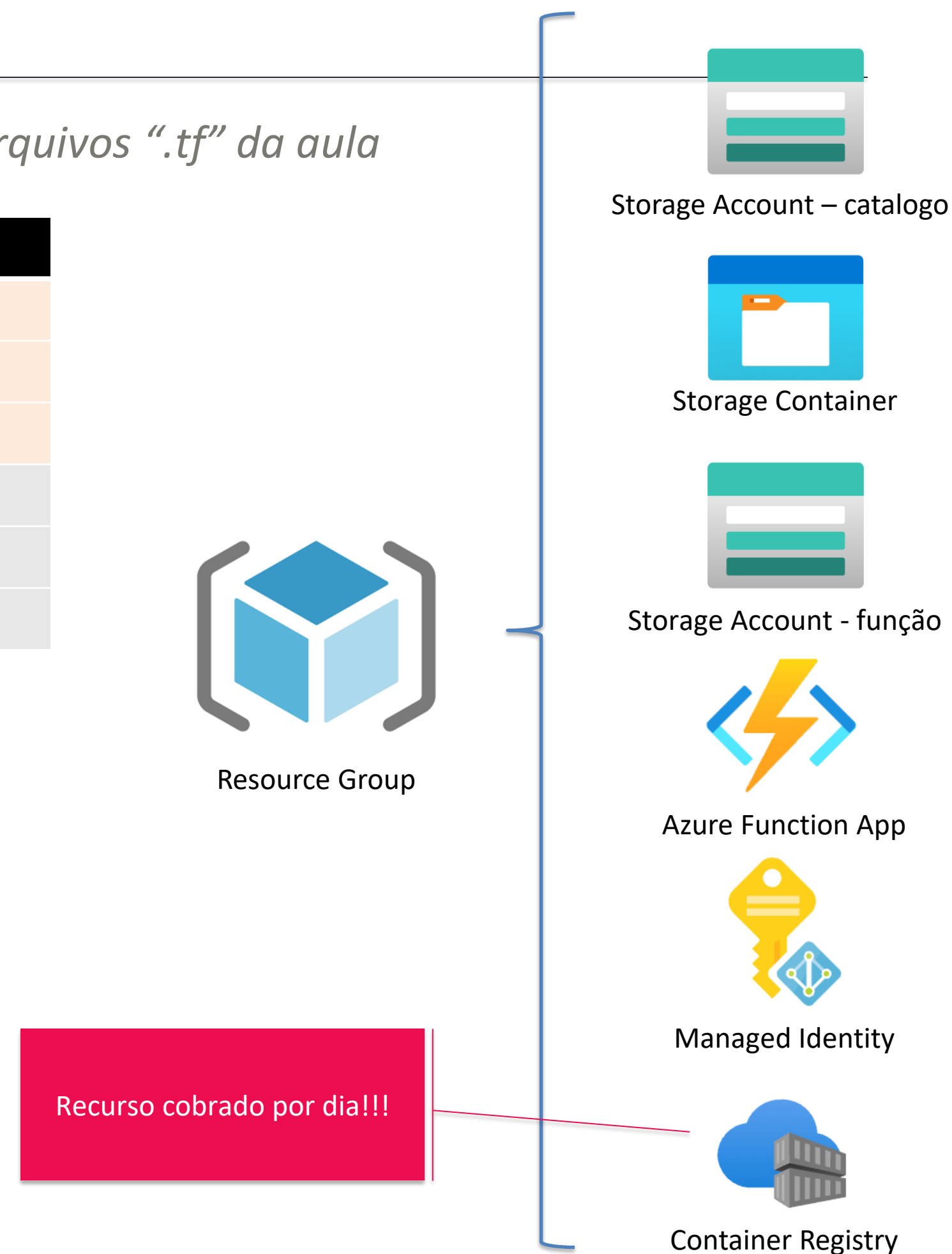
2 – Enquanto o terraform trabalha, vamos explorar o que temos nos arquivos “.tf” da aula

aie-cloud/aulas/03-serverless-containers/lab/terraform

Arquivo	O que define
main.tf	Providers, sufixo aleatório, Resource Group, locals
variables.tf	location e aci_enabled (para criar o ACI posteriormente)
outputs.tf	Nomes e endpoints consumidos pelos scripts Python
storage.tf	2 Storage Account + 1 containers (catalogo) + lifecycle
function.tf	Function App + Managed Identity + role Blob Data Reader
containers.tf	ACR + UAI + role + ACI (count condicional)

Consultar recursos criados pelo terraform

terraform output

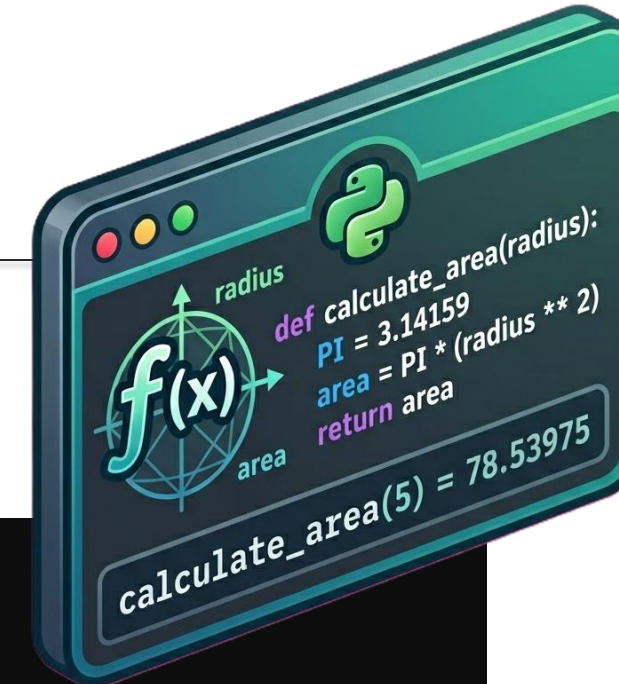


Avaliação do arquivo Python

- Localize o diretório function/v1-mock usando o VS Code;
- Explore o arquivo function_app.py, veja seu conteúdo;
- Identifique as funções
 - listar_produtos
 - health
- Explore os demais arquivos do diretório

Breve explicação:

O que são os “Decorators”



```
import json
import logging
import azure.functions as func

app = func.FunctionApp(http_auth_level=func.AuthLevel.ANONYMOUS)

# Mock inicial — depois substituído pelo Blob na v2-blob/
PRODUTOS MOCK = [
    {"id": 1, "nome": "Cadeira Ergonômica DXRacer",
     "categoria": "moveis", "preco": 1499.90},
    ...
]

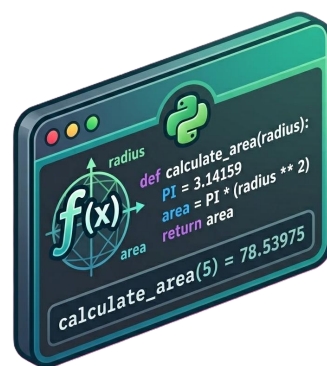
@app.route(route="produtos", methods=["GET"])
def listar_produtos(req: func.HttpRequest) -> func.HttpResponse:
    # Início da função
    ...
    # Retorno da função
    return func.HttpResponse(
        json.dumps({"total": len(resultado),
                    "produtos": resultado}, ensure_ascii=False),
        mimetype="application/json",
        status_code=200,
    )

@app.route(route="health", methods=["GET"])
def health(req: func.HttpRequest) -> func.HttpResponse:
    return func.HttpResponse(
        json.dumps({"status": "ok", "service": "qc-catalogo",
                    "source": "mock"}),
        mimetype="application/json",
    )
```


Faremos o “Deploy” deste código para o Function App

```
# Pegar o nome da Function App
FUNC_NAME=$(cd ~/aie-cloud/aulas/03-serverless-containers/lab/terraform && terraform output -raw
function_app_name)
echo "Function: $FUNC_NAME"

# Deploy da v1 (mock)
cd ~/aie-cloud/aulas/03-serverless-containers/lab/function/v1-mock
func azure functionapp publish "$FUNC_NAME" --python
```



v1-mock

(arquivos .py, requirements.txt, etc.)



Azure Function App

⚠ ESPERADO, NÃO É FALHA

Cold start de 2–3s

A **primeira** chamada demora — a função precisa acordar. As seguintes são milissegundos.

✓ CHECKPOINT L₁

O curl retorna JSON com a lista de produtos da Function deployed?

Dica:

Pelas URLs exibidas no terminal, acesse sua função usando o browser. E pelo celular?

(Variação com parametro ?categoria=moveis)

Quem consegue acessar por curl?

Refaça o laboratório, e acompanhe na **interface gráfica** o que está acontecendo.

02

Triggers & Managed Identity

Cold start em profundidade e o coração da Aula 3: a Function lê o arquivo Blob sem uma única credencial no código .



· A VACINA ·

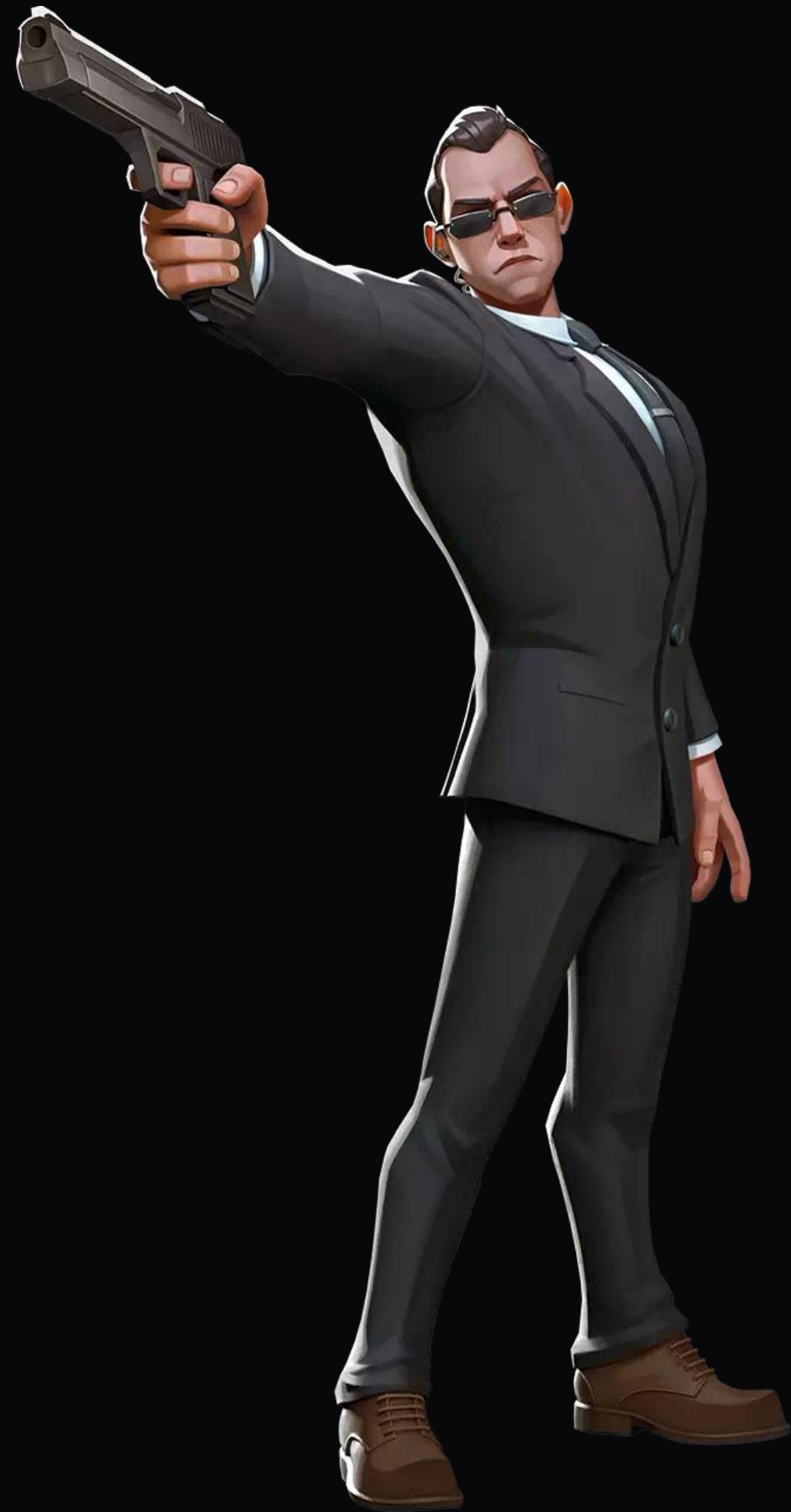
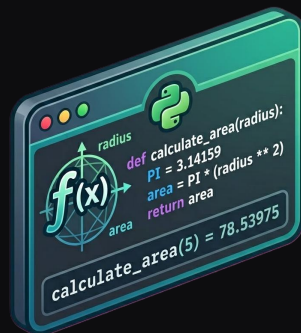
Agente em produção não usa **connection string.**

A senha no código vaza no primeiro push.

O antídoto tem nome: RBAC e Managed Identity.

A função tem identidade própria, você dá uma role — e o token vem sozinho em runtime.

Você nunca vê, nunca digita e nunca commita a credencial.



Tire a credencial do código

Princípio do menor privilégio aplicado a agentes

✗ ERRADO

Credencial no código

A chave está ali, em texto — pronta pra vazar no Git, no log, no incident.

```
blob = BlobServiceClient(url, credential="account-key-abc-123...")
```

✓ CERTO

Managed Identity

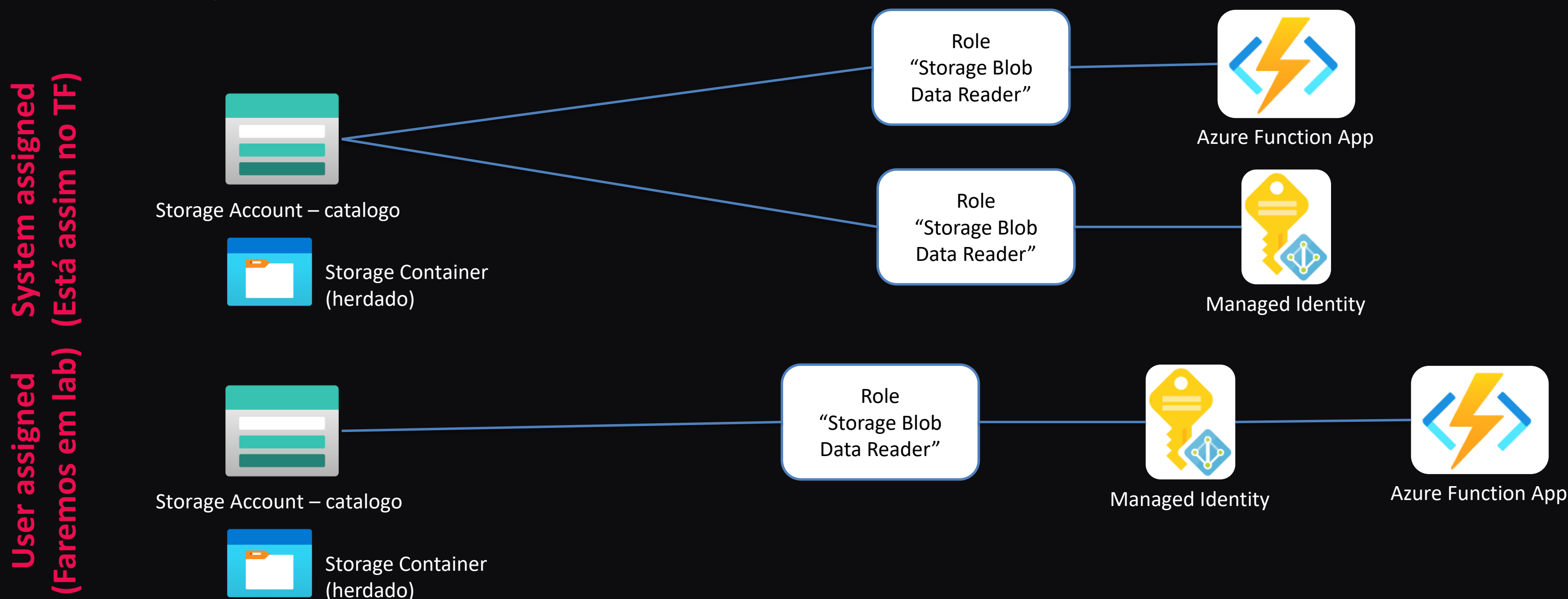
A função tem identidade no Entra ID. O SDK pega o token sozinho — sem credencial em lugar nenhum.

```
cred = DefaultAzureCredential()  
blob = BlobServiceClient(url, credential=cred)
```

**Cada agente em produção tem a própria Managed Identity : busca só lê produtos, pedidos só escreve pedidos.
Menor privilégio + auditoria por identidade.**

Explore no portal, o “catálogo” dentro da Storage Account.
Verifique o controle de acessos (IAM) quem tem acessos de leitura – Role Assignments.

Os acessos podem vir em dois sabores:



Onde mora o segredo define o risco

ESTRATÉGIA	ONDE GUARDA O SEGREDO	QUEM ACESSA	RISCO
Hardcoded	Código (Git)	Quem tem o repo	⚠ Vazamento global
Variável de ambiente	App Settings	Quem acessa o portal	Médio
Key Vault + chave	Vault	App, via chave do Vault	Seguro, mas ainda há chave
Managed Identity	Nenhum lugar	Só a Function associada	Mínimo ✓
Role Assign direto	Nenhum lugar	Só a Function autorizada	Mínimo ✓

Pergunta pró:

para quais dessas um vazamento do código no GitHub ainda seria problema?

Vazou o código. O que muda?

O TESTE QUE GENERALIZA

“Se este arquivo virar público agora, o atacante consegue se autenticar?”

SÓ UMA LINHA ENTREGA A CREDENCIAL

Hardcoded

a senha está dentro do arquivo que vazou

SIM

Env var · Key Vault ref

vaza o nome da variável, não o valor

NÃO

Managed Identity

não existe valor algum para vaziar

NÃO

O asterisco do Key Vault: só é seguro se a chave que abre o cofre não estiver no código. Se estiver, é hardcoded com passo extra — e entrega o cofre inteiro em vez de uma senha.

MAS O CÓDIGO VAZADO AINDA ENTREGA

● Reconhecimento

Nomes de storage, vault e SQL Server — todos endereçáveis publicamente. O atacante testa o alvo certo em vez de varrer a internet.

● Lógica e falhas

Onde a autorização é validada, SQL injection, IDOR. Tudo legível.

● O teto do estrago

Quais roles a Function tem. Se ela é Owner no RG, comprometê-la vale mais do que valer uma senha de banco.

Com Managed Identity o atacante deixa de roubar a credencial: ele precisa comprometer o processo que a possui. É por isso que least privilege continua importando — ele define o teto do estrago.

Procure por "password" — não vai achar

O próximo Lab irá retornar a lista de produtos que está armazenada no Storage.

Antes de prosseguir, avalie o arquivo “function_app.py”

A credencial nunca existe no código pra vazar. O SDK negocia o token via Managed Identity em runtime.

```
# Identidade da Function — nenhuma chave, nenhuma string (Código simplificado)

credential = DefaultAzureCredential()

blob = BlobServiceClient(f"https://{STORAGE_ACCOUNT}.blob.core.windows.net", credential) # Baixa o produtos.csv do Blob da Aula 2 e vira JSON

def carregar_produtos():
    client = blob.get_blob_client("catalogo", "produtos.csv" )

    csv_text = client.download_blob().readall().decode( "utf-8" )

    return list(csv.DictReader(csv_text.splitlines())) # GET /api/produtos?categoria=moveis → 20 produtos reais ✓
```

Pergunta para a turma:

Se um agente precisar acessar 5 storages diferentes, qual a estratégia?

Um agente, cinco storages

O PRINCÍPIO

Identidade responde **quem é**. Role assignment responde **o que pode**. São eixos diferentes — não se resolve um multiplicando o outro.

AS DUAS RESPOSTAS ERRADAS

5 identidades, uma por storage

Confunde os dois eixos. O processo é um só — se cai, o atacante leva as cinco credenciais junto.

Um role no escopo do Resource Group

Já concede o storage nº 6, que ainda não existe. É permissão que cresce sozinha.

A RESPOSTA

Uma user-assigned Managed Identity + cinco role assignments, cada um com escopo no ID de um storage e o papel mínimo daquele acesso — Reader onde só lê, Contributor onde escreve.

O escopo desce até o container:

```
.../blobServices/default/containers/vendas
```

No código não muda uma linha: o mesmo DefaultAzureCredential e o mesmo BlobServiceClient atendem as cinco contas. A autorização acontece no plano de dados do Azure — é por isso que escala.

QUANDO SEPARAR A IDENTIDADE

Não é pelo número de storages — é por blast radius. Duas capacidades do agente que você revogaria em momentos diferentes são duas identidades.

O teste: “eu revogaria um sem revogar o outro?”

ONDE A RECEITA QUEBRA

● Storages em outro tenant

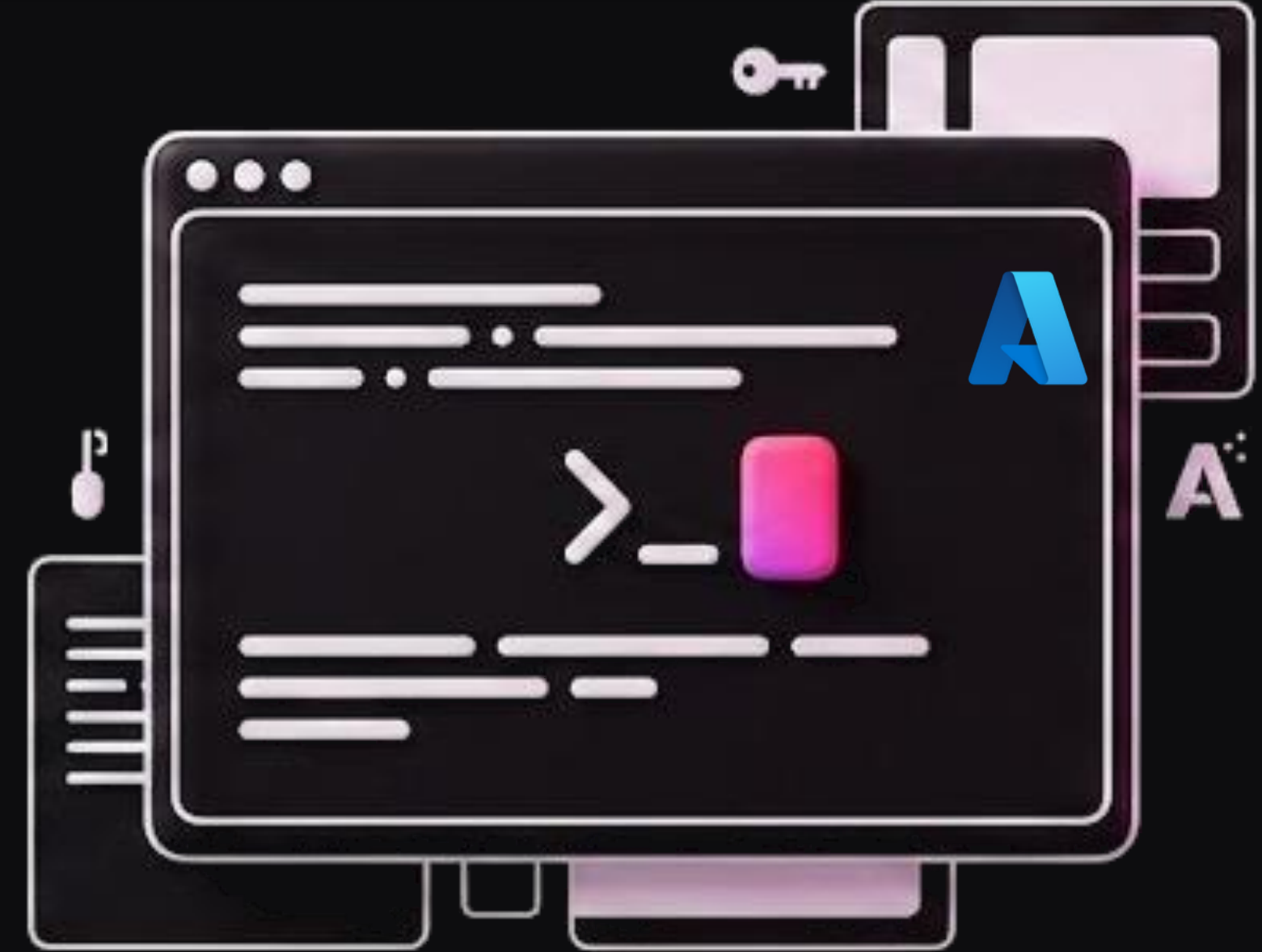
Managed Identity não atravessa tenant. Aí é Service Principal com Workload Identity Federation.

● Dezenas de contas

Atribuição por recurso desanda. Escopo maior com condition ABAC por tag ou prefixo de path.

Nunca: cinco connection strings no Key Vault. A chave da conta não tem granularidade — quem a tem apaga tudo.

LAB 2



Function Lê o arquivo via Managed Identity

Antes de continuar, certifique-se de ter à disposição:

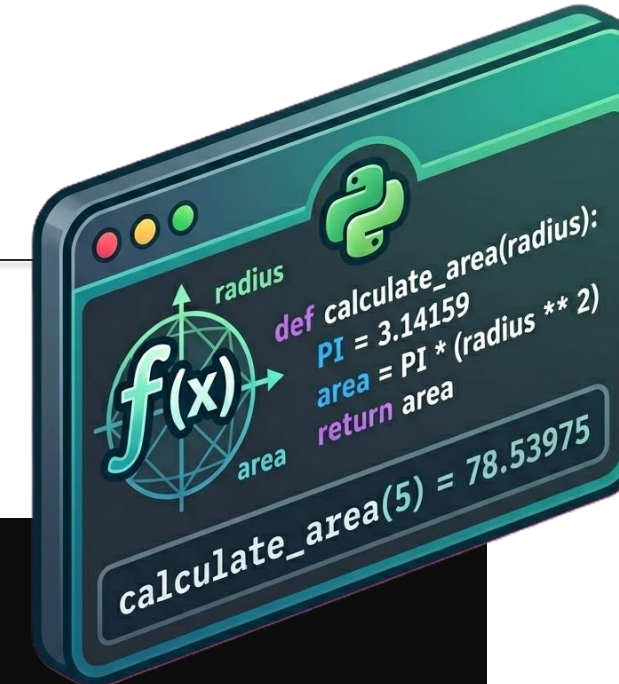
- Conta Azure já logada
- Cloud Shell funcionando
- Uma boa xícara de café ☕

Avaliação do arquivo Python

- Localize o diretório function/v2-blob usando o VS Code;
- Explore o arquivo function_app.py, veja seu conteúdo;
- Identifique as funções
 - listar_produtos
 - health
- Explore os demais arquivos do diretório

Breve explicação:

O que são os “Decorators”



```
import json
import logging
import azure.functions as func

app = func.FunctionApp(http_auth_level=func.AuthLevel.ANONYMOUS)

# Mock inicial — depois substituído pelo Blob na v2-blob/
PRODUTOS MOCK = [
    {"id": 1, "nome": "Cadeira Ergonômica DXRacer",
     "categoria": "moveis", "preco": 1499.90},
    ...
]

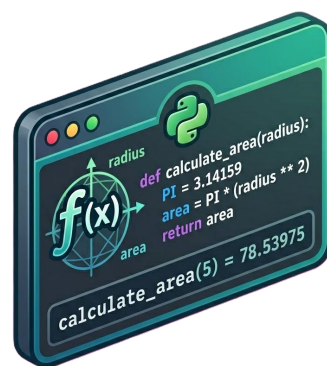
@app.route(route="produtos", methods=["GET"])
def listar_produtos(req: func.HttpRequest) -> func.HttpResponse:
    # Início da função
    ...
    # Retorno da função
    return func.HttpResponse(
        json.dumps({"total": len(resultado),
                    "produtos": resultado}, ensure_ascii=False),
        mimetype="application/json",
        status_code=200,
    )

@app.route(route="health", methods=["GET"])
def health(req: func.HttpRequest) -> func.HttpResponse:
    return func.HttpResponse(
        json.dumps({"status": "ok", "service": "qc-catalogo",
                    "source": "mock"}),
        mimetype="application/json",
    )
```

Faremos o “Deploy” deste código para o Function App

```
# Pegar o nome da Function App
FUNC_NAME=$(cd ~/aie-cloud/aulas/03-serverless-containers/lab/terraform && terraform output -raw
function_app_name)
echo "Function: $FUNC_NAME"

# Deploy da v2 (blob)
cd ~/aie-cloud/aulas/03-serverless-containers/lab/function/v2-blob
func azure functionapp publish "$FUNC_NAME" --python
```



v2-blob

(arquivos .py, requirements.txt, etc.)



Azure Function App

⚠ ESPERADO, NÃO É FALHA

Cold start de 2–3s

A **primeira** chamada demora — a função precisa acordar. As seguintes são milissegundos.

✓ CHECKPOINT L₁

O curl retorna JSON com a lista de produtos da Function deployed?

Dica:

Pelas URLs exibidas no terminal, acesse sua função usando o browser. E pelo celular?

(Variação com parametro ?categoria=moveis)

Quem consegue acessar por curl?

Refaça o laboratório, e acompanhe na **interface gráfica** o que está acontecendo.

Alteração para Managed Identity



Edite com o professor o código python para usar o Client ID da sua Identidade Gerenciada

Type	: Microsoft.ManagedIdentity/userAssignedIdentities
Client ID	: aea077bd-2871-40ad-bd65-6e22946d5281
Object (principal) ID	: 21b8cbb9-c74e-48a1-a779-66a49fa13d7a
Isolation Scope	: None

```
DefaultAzureCredential(managed_identity_client_id='OBTER ou passar por variável de ambiente')
```

Alternativa: Exercício para casa

```
DefaultAzureCredential(managed_identity_client_id=os.environ["USER_ASSIGNED_CLIENT_ID"])
```

Salve o arquivo e faça novo Deploy, após essa alteração.

Pontos para discutir com a turma

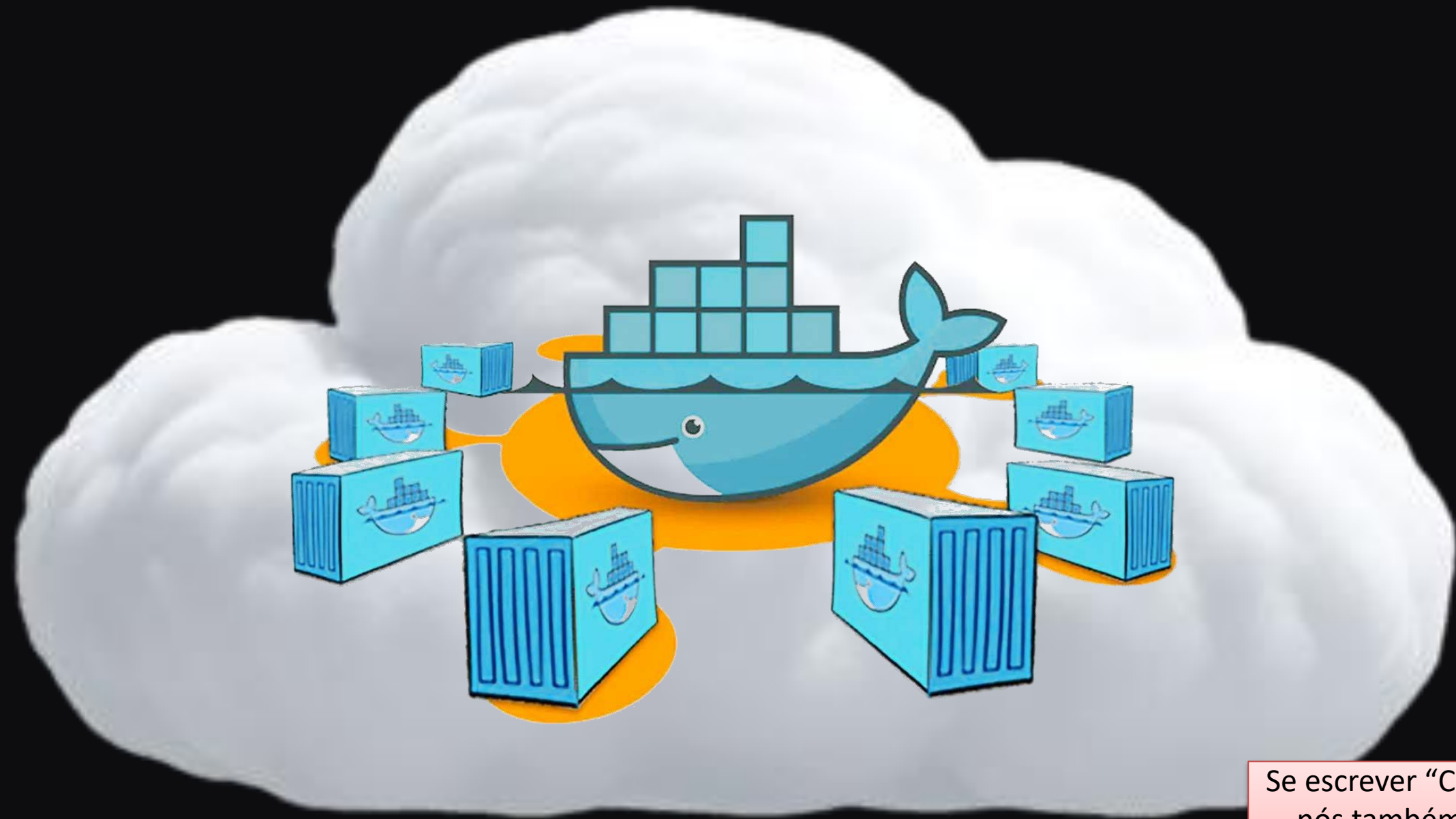
- Cold Start
- Decorators
- Identity
- Como a Function consegue ler sem credenciais?
- Se um agente precisar acessar 5 storages diferentes, qual a estratégia?
- Explorar com a turma o acesso do Storage direto para função e com acesso via Managed Identity
 - System assigned vs User assigned

```
DefaultAzureCredential(managed_identity_client_id='OBTER ou passar por variável de ambiente')
```

03

Contêineres

Docker, ACR e ACI — e o mesmo código Python da Function, empacotado e rodando num container.



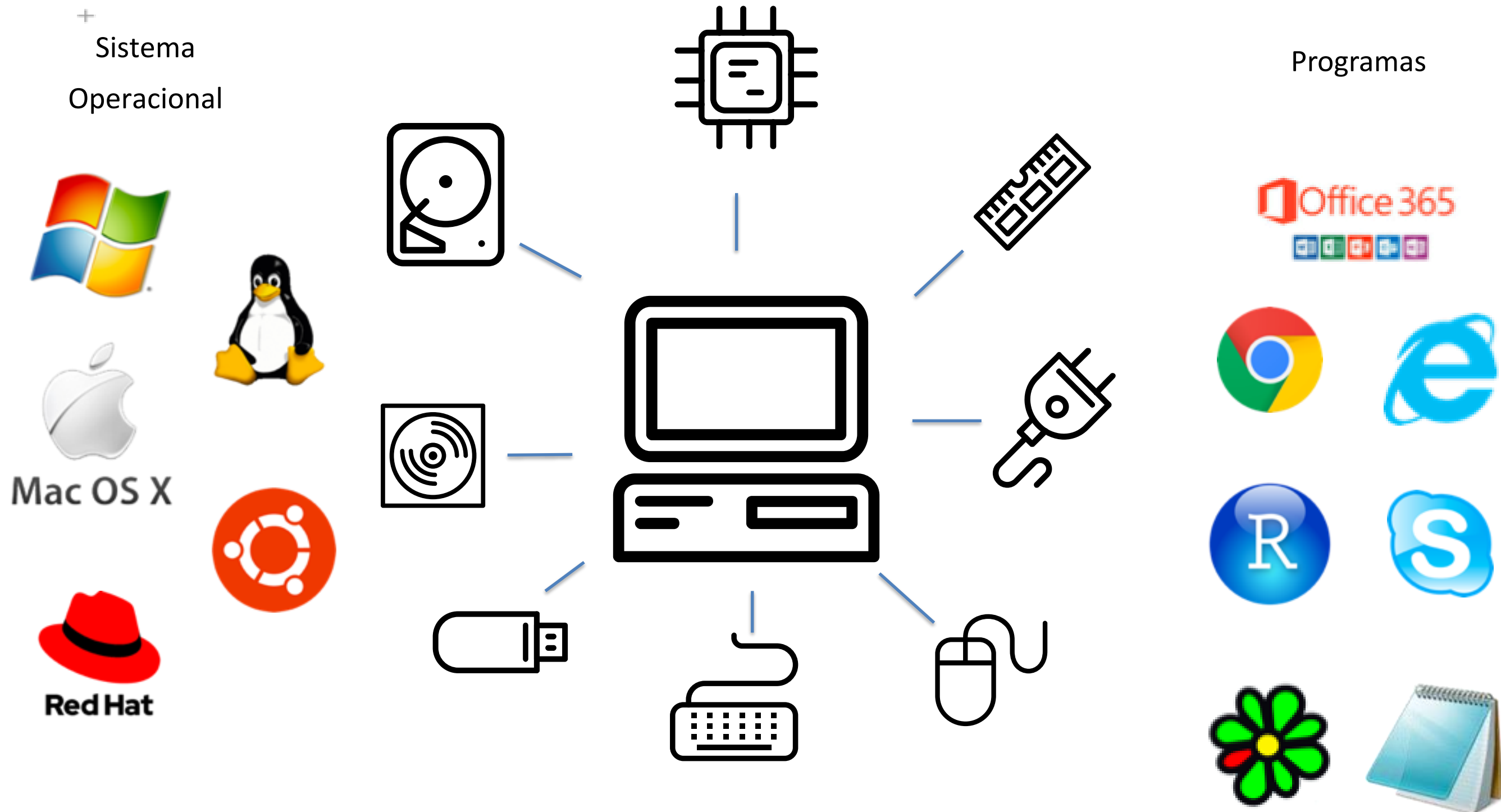
Se escrever “Container”,
nós também vamos
entender.



Introdução “Máquinas Virtuais”

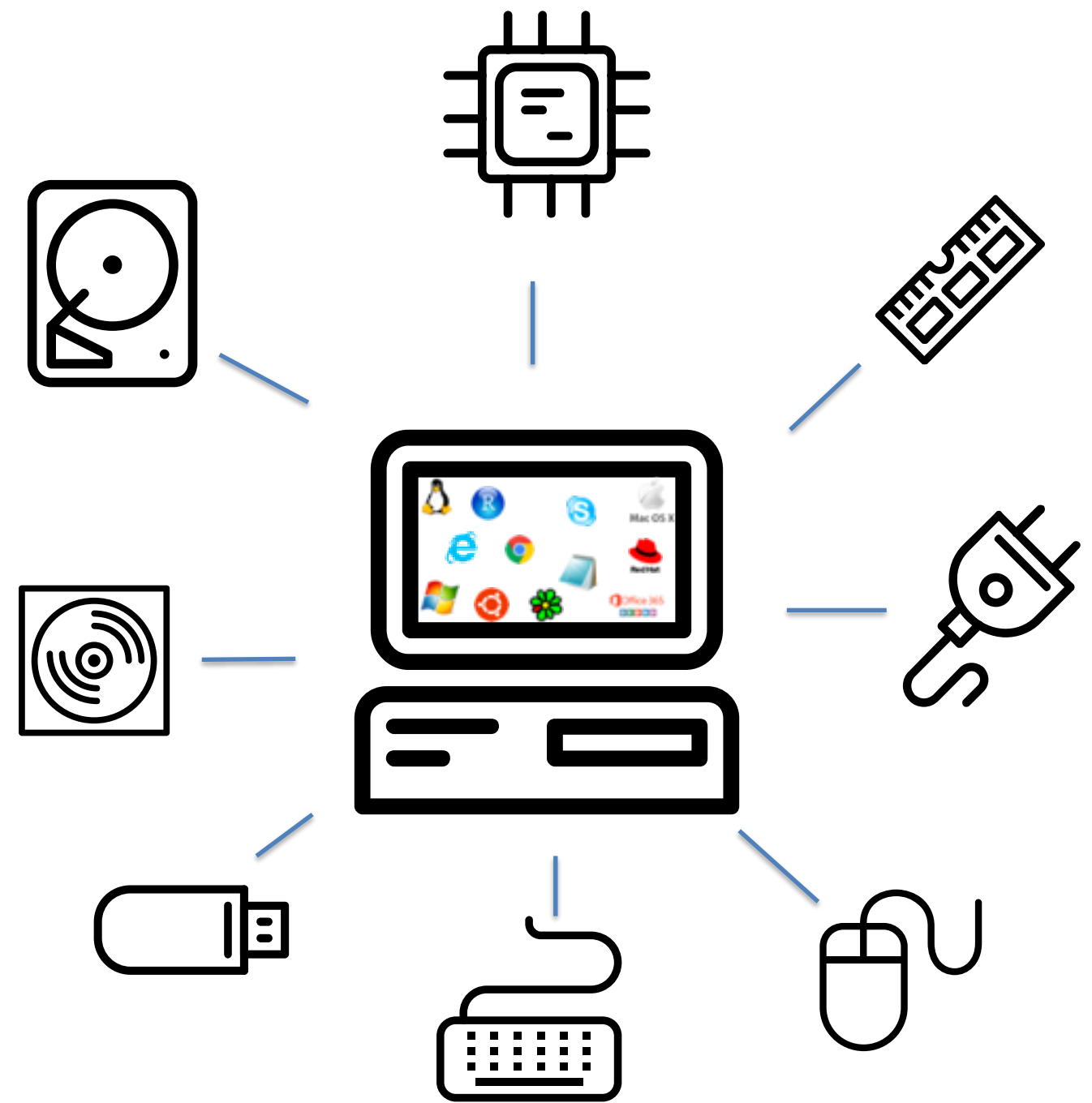
O que o seu computador precisa para você utilizá-lo?

Retomando o assunto da VM



Introdução “Máquinas Virtuais”

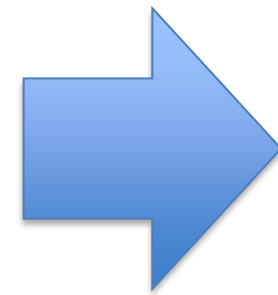
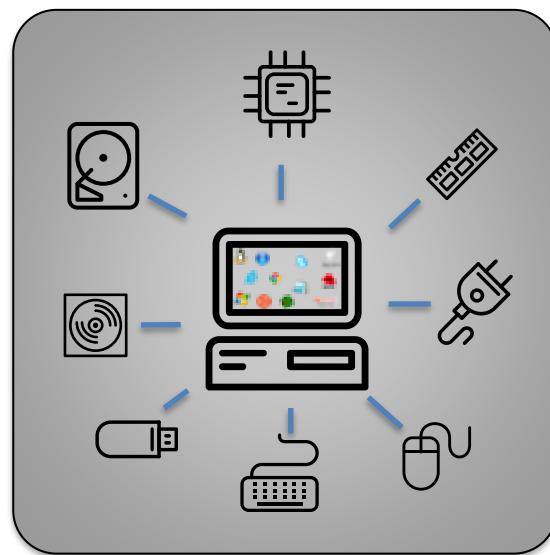
Retomando o assunto da VM



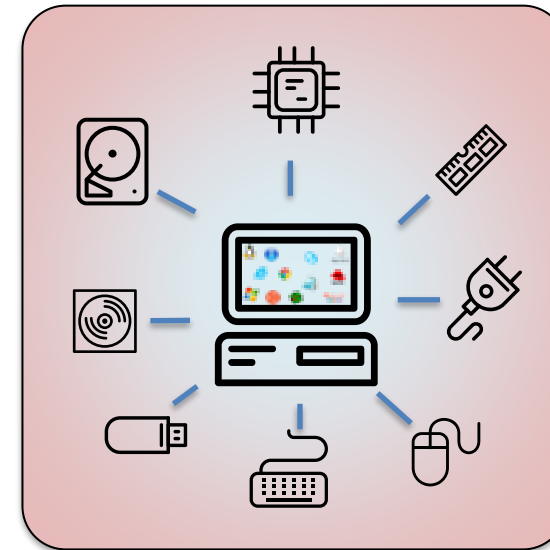
Introdução “Máquinas Virtuais”

Retomando o assunto da VM

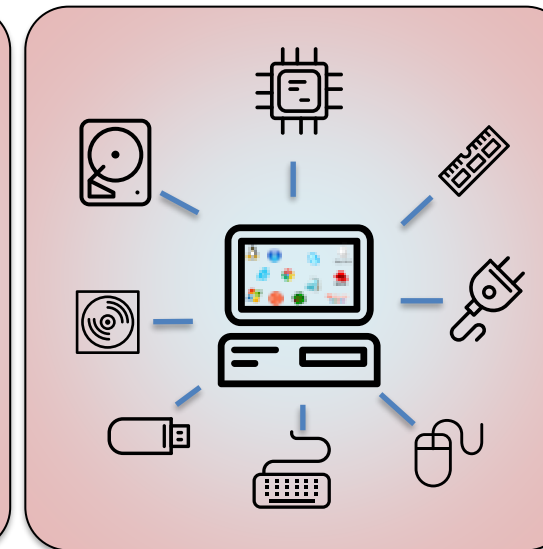
Imagem (Template)
de Máquina Virtual



VMs
Imagens em execução

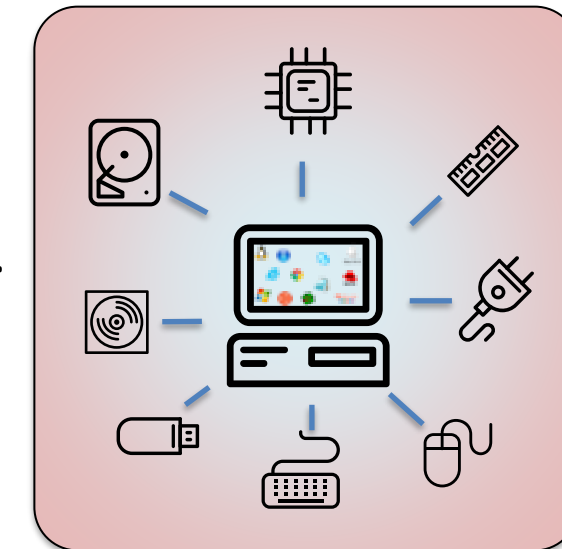


Máquina 1



Máquina 2

...

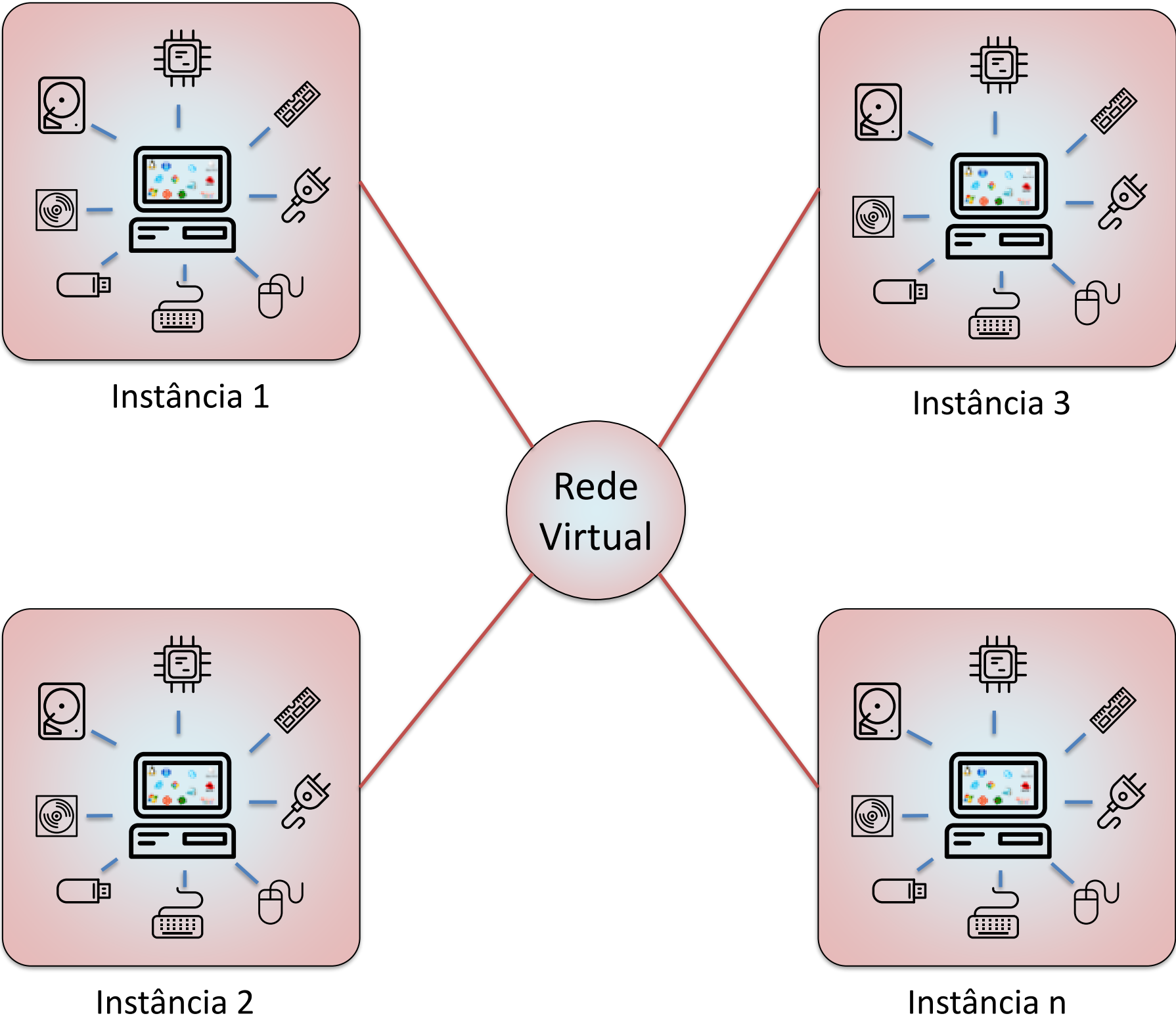


Máquina n

Introdução “Máquinas Virtuais”

As VMs em execução, podem rodar softwares diferentes

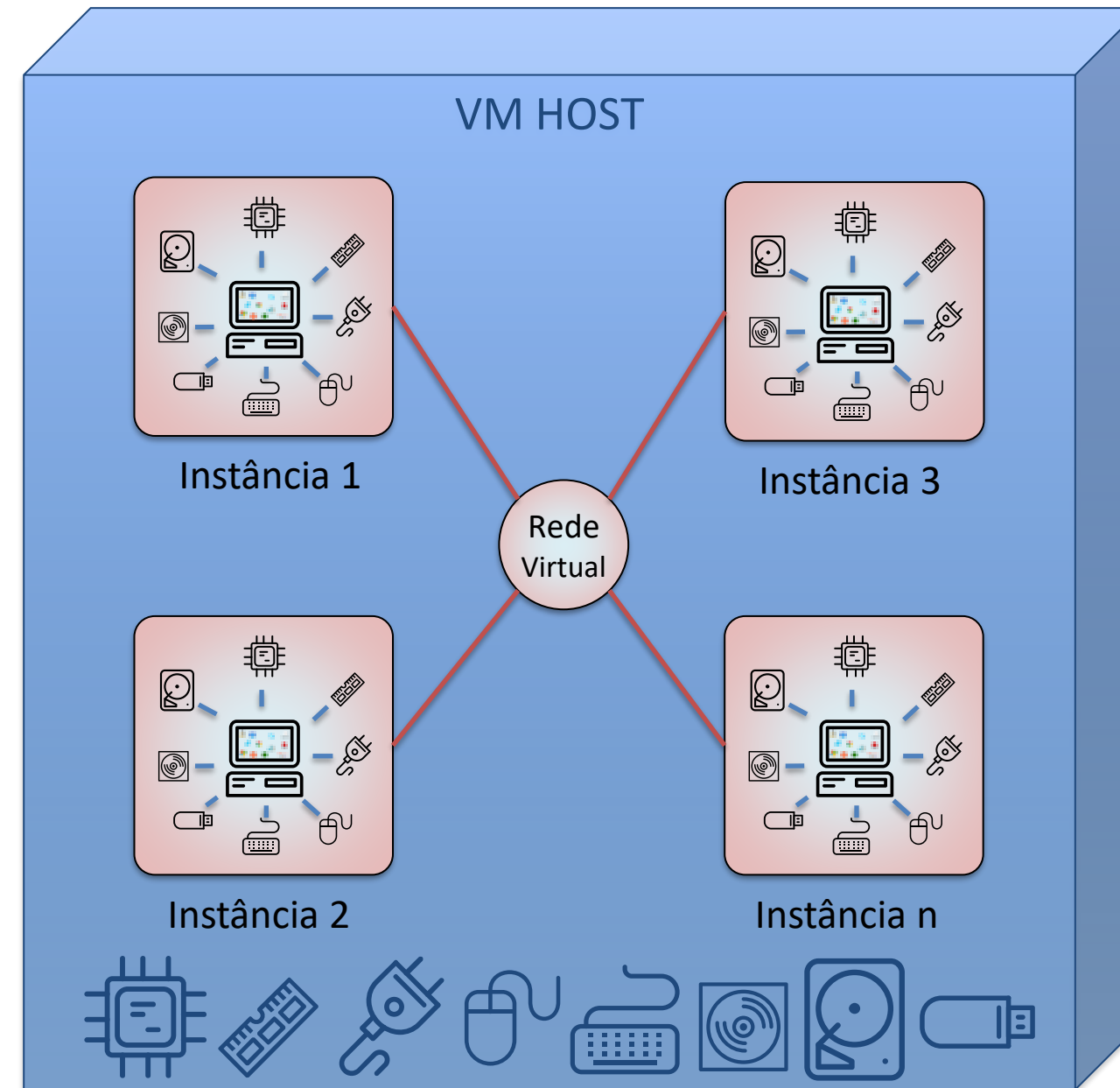
Retomando o assunto da VM



Introdução “Máquinas Virtuais”

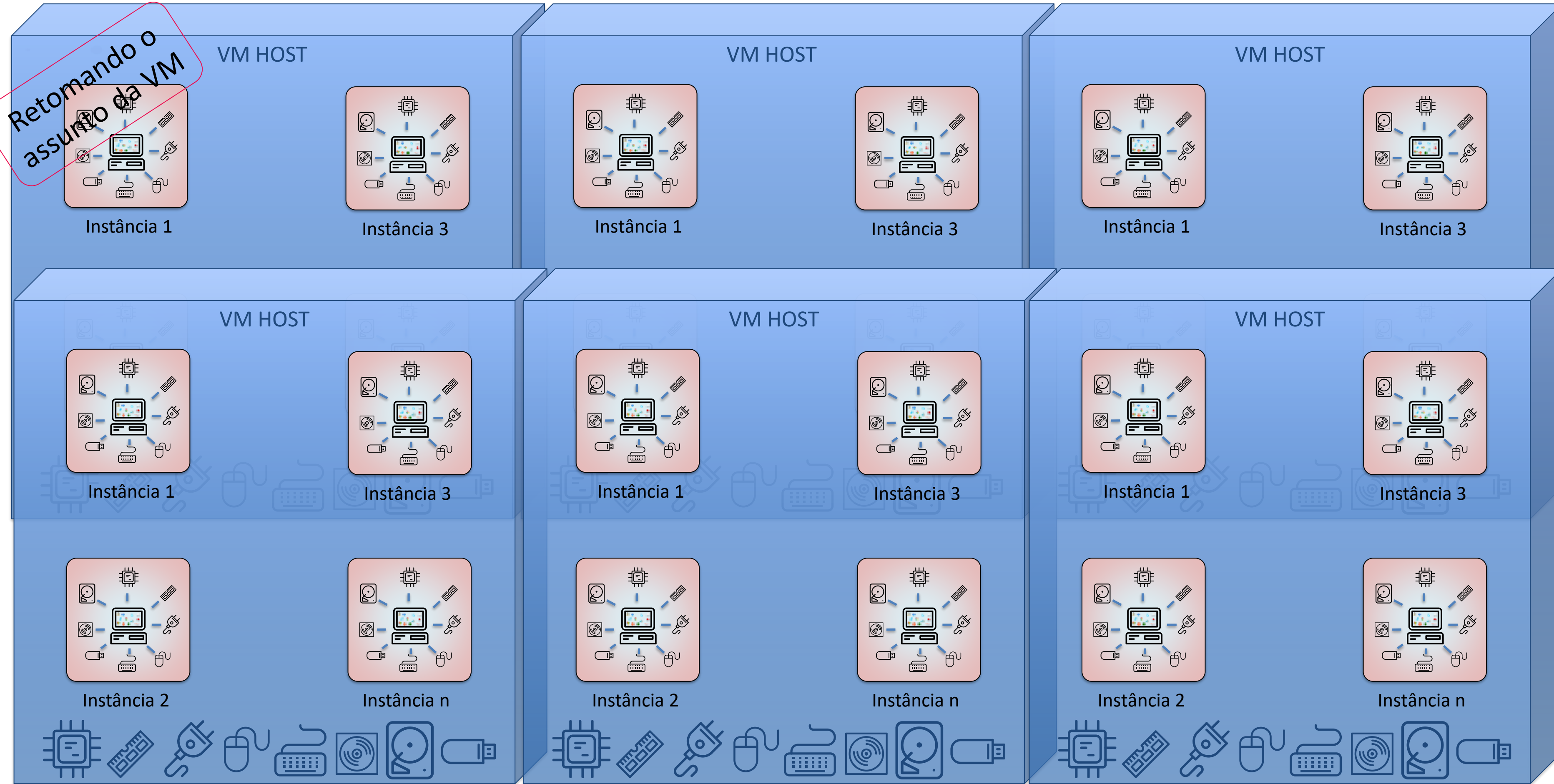
E esses VMs, utilizam os recursos de algum HOST

Retomando o assunto da VM



Introdução “Máquinas Virtuais”

Em um Datacenter, as VMs ficam distribuídas em vários Hosts



► Um Datacenter por dentro

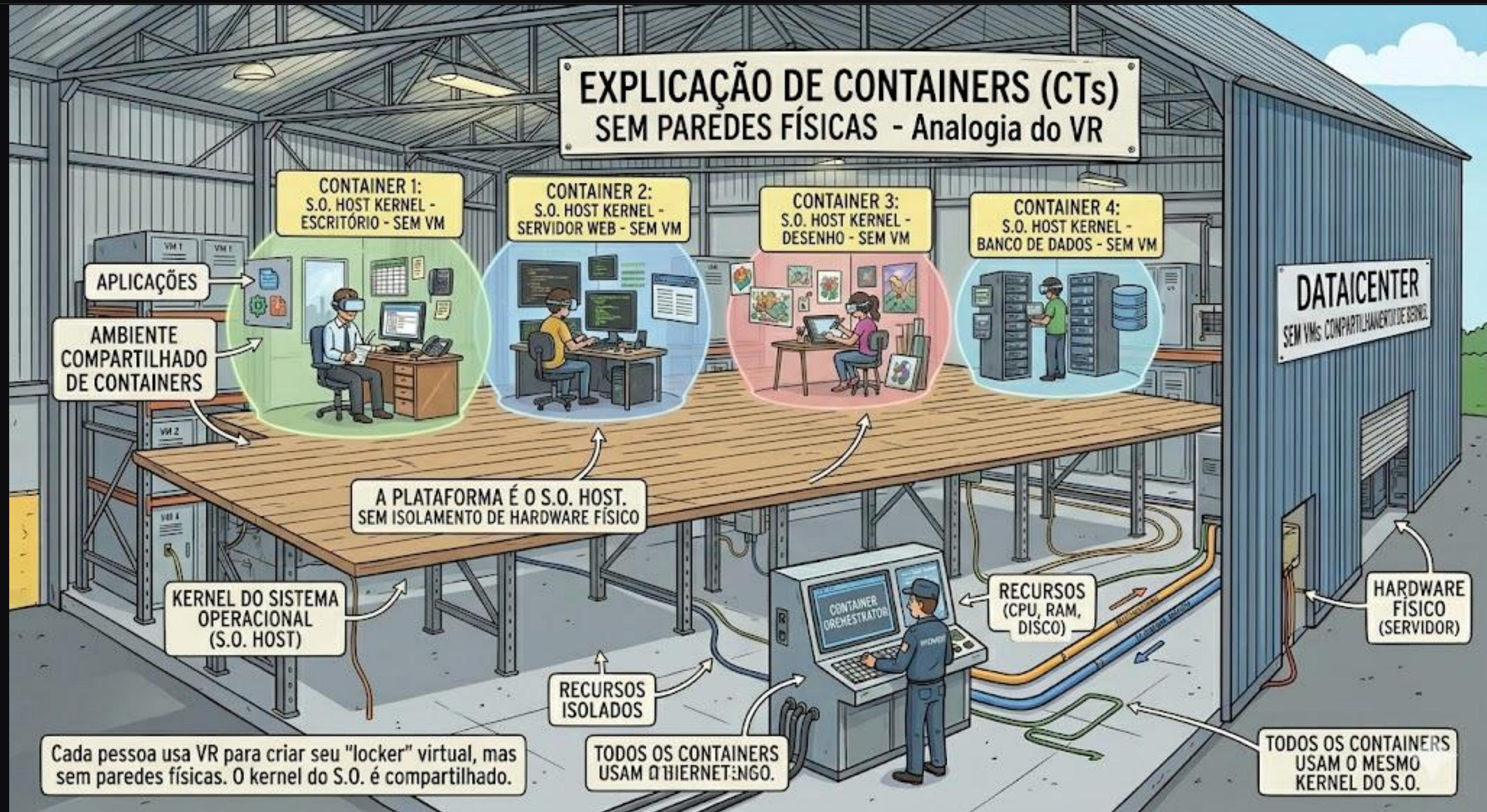


► MÁQUINAS VIRTUAIS



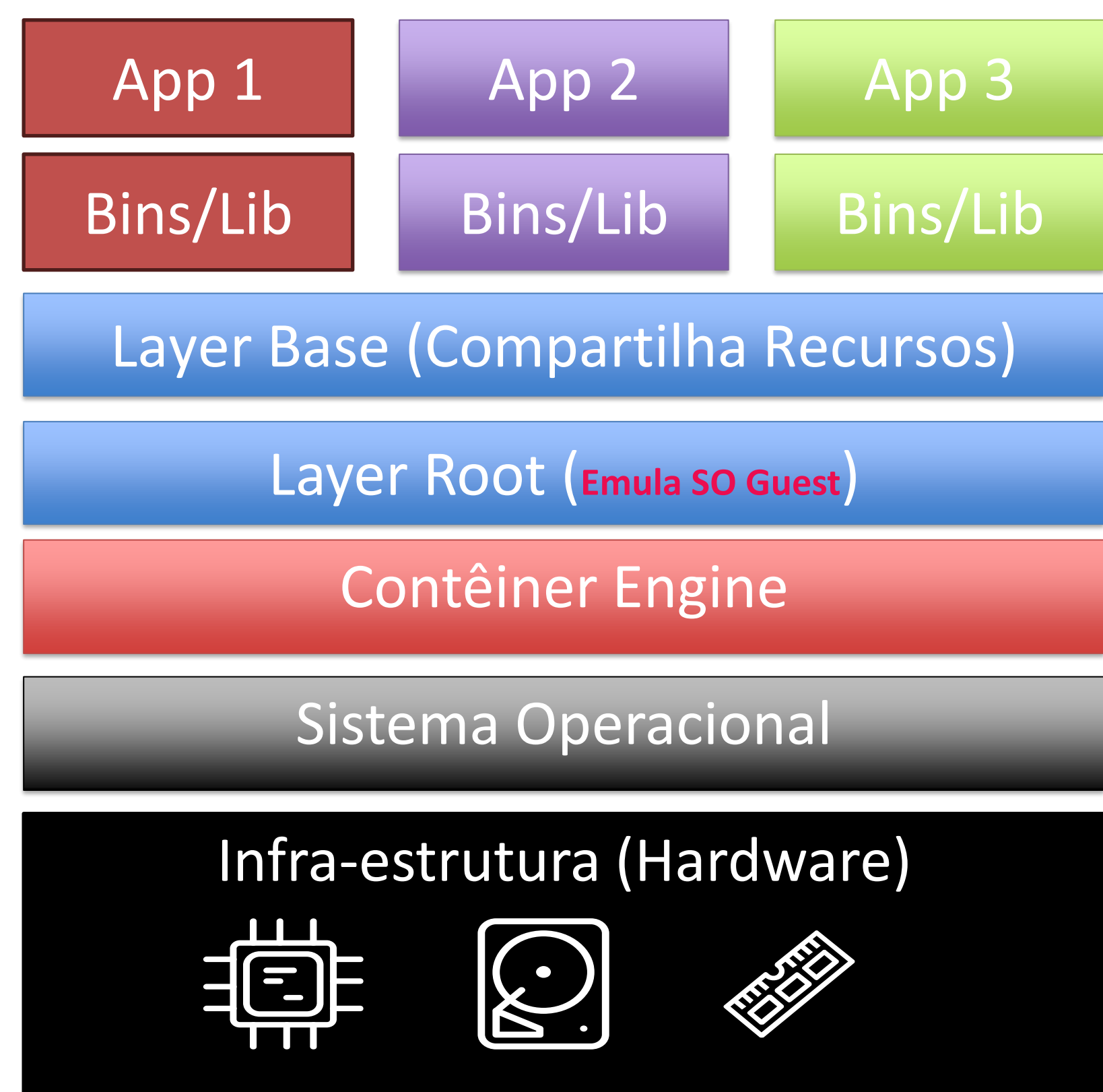
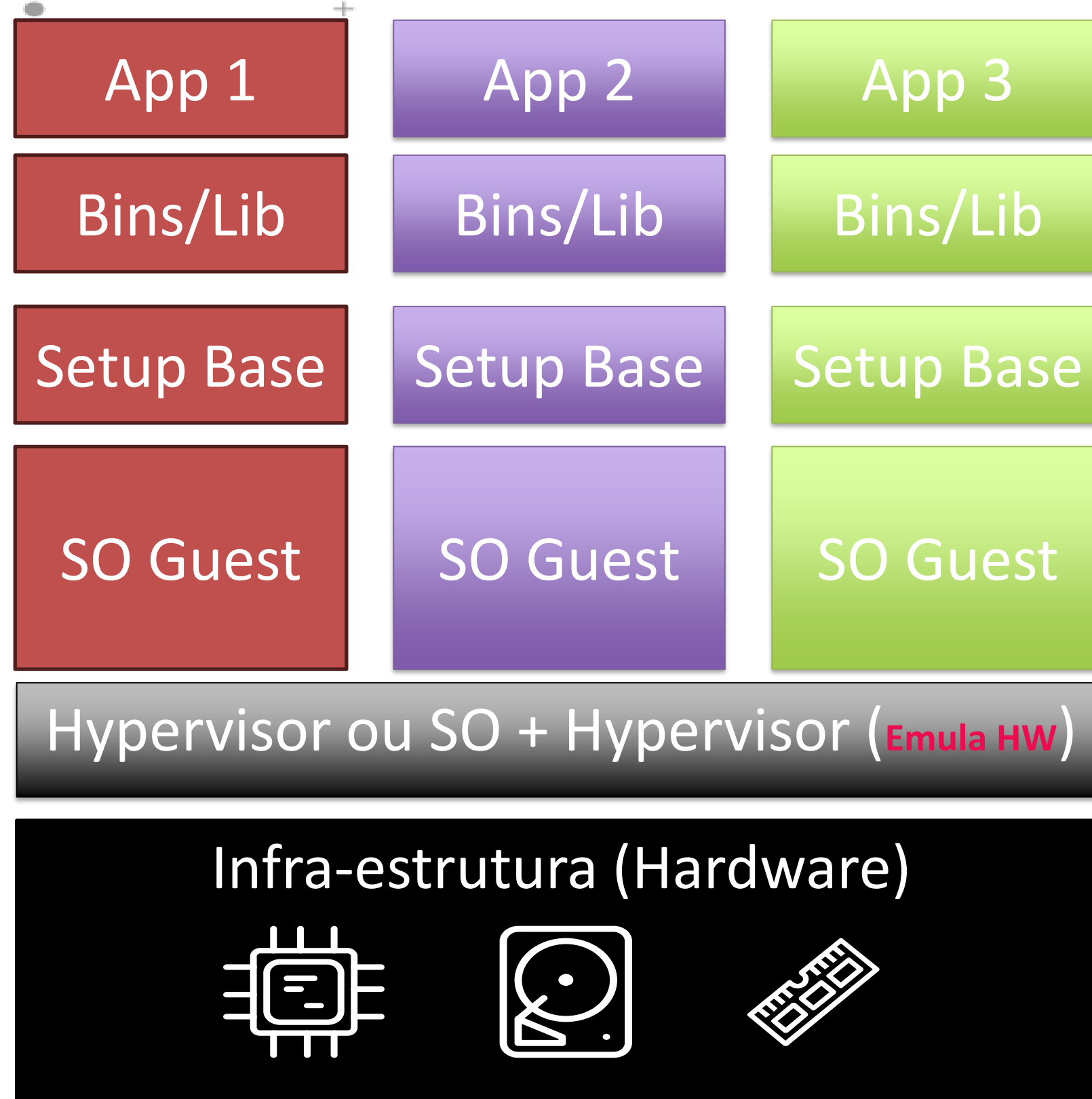
VMs, analogia de lockers dentro de um HOST (Galpão) – Cada pessoa é um Software em VM

► CONTÊINERES



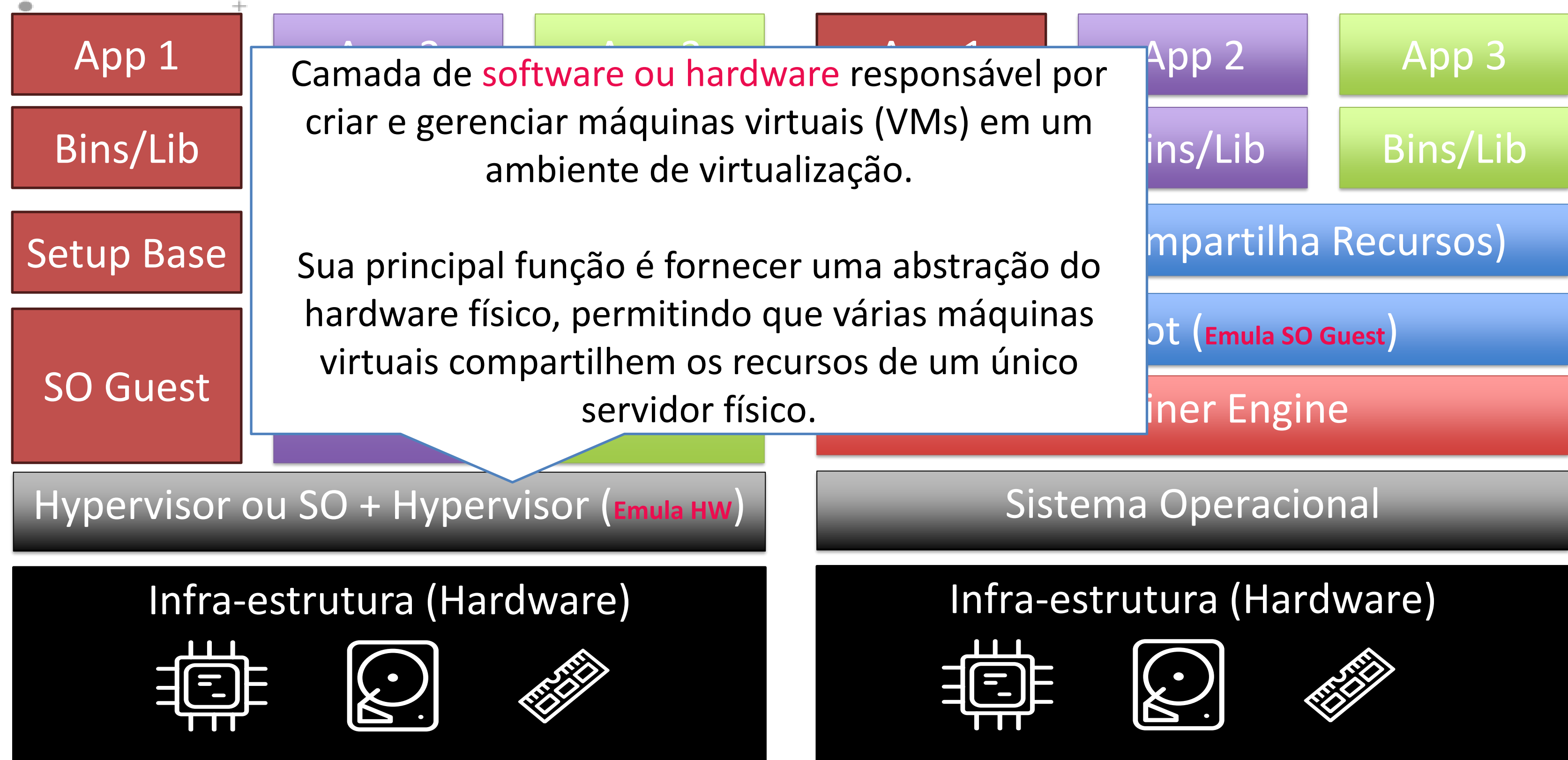
CTs, analogia de VR dentro de um HOST (Galpão) – Cada pessoa é um Software em Container

Máquinas Virtuais vs Contêineres



- SO = Sistema Operacional

Máquinas Virtuais vs Contêineres



- SO = Sistema Operacional

Container : principais conceitos

DOCKERFILE

A receita



Instruções pra construir a imagem, linha por linha.

IMAGEM

Template imutável



SO + dependências + código, congelado num artefato.

LAYERS

Camada reaproveitável



Cada RUN/COPY é uma camada. A ordem importa.

REGISTRY

ACR



O depósito (repositório) de imagens.

ACR (Azure Container Registry) é o "Docker Hub" privado da empresa, na Azure.

CONTAINER

Imagem rodando



A instância em execução da imagem.

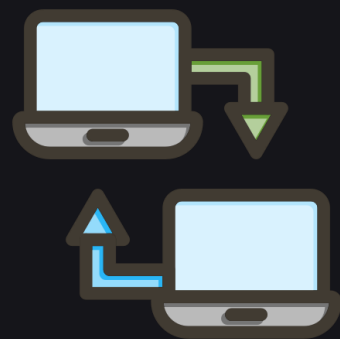
O ciclo:

escreve Dockerfile → builda imagem → push pro registry → roda container.

"Na minha máquina funciona" — resolvido

PORTABILIDADE

Roda igual



Portabilidade: roda igual no laptop, no servidor, na nuvem — acabou o famoso na minha máquina funciona.

PACKING

Tudo embalado



Packing de dependências: Python 3.11 mais 30 libs mais binários, tudo embalado num artefato só.

ISOLAMENTO

Sem conflito



Isolamento: dois apps na mesma máquina não brigam por versão de biblioteca.

PADRÃO

~100% das stacks



Padrão da indústria: praticamente 100% das stacks modernas usam container.

Mesmo código, qual deploy?

CENÁRIO	FUNCTION	CONTAINER
Linguagem Python / JS / .NET	✓	✓
Rust / Go / Java legado	⚠ custom handler	✓
Dependências de sistema (apt, libs nativas)	✗	✓
Código pequeno, orientado a evento	✓	⚠ overhead
Custo intermitente / constante	✓ por chamada	✓ por segundo

Nós pegaremos o mesmo código Python da Function e empacotamos num container.
Mesmo comportamento esperado, deploy diferente — vocês decidem qual leva pra produção.

ACI: container sem orquestrador (Host)

A forma mais simples de rodar um container na nuvem



Azure Container Instance

Sem Host, sem cluster, nem health check ...
Sobe um container e pronto!

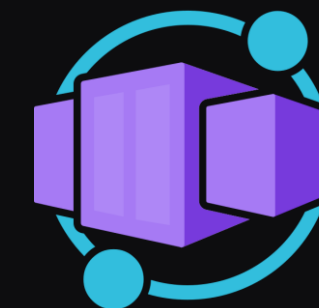
Ideal para cargas pontuais e simples



Azure Container Registry

Armazena as imagens de container

Containers em Outros sabores



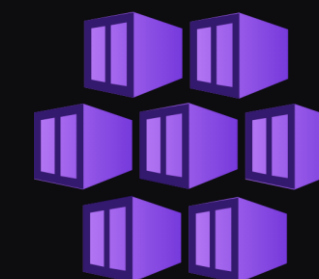
Container Apps

Escalabilidade automática e segura
Ideal para aplicações modernas e contínuas



Container Apps Jobs

Similar ao Container Apps, para processamentos Batch



Azure Kubernetes Services (AKS)

Controle avançado de clusters e rede
Ideal para cenários complexos e corporativos

LAB 3



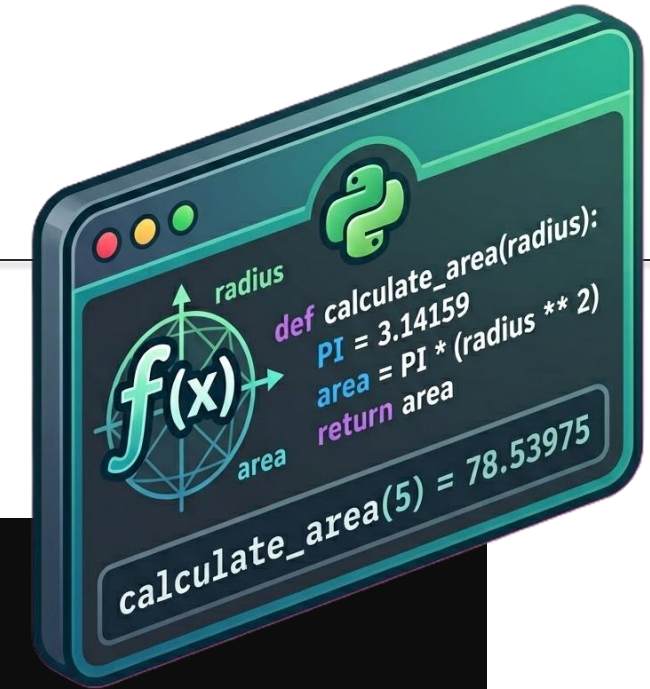
Containerização + ACR + ACI

Antes de continuar, certifique-se de ter à disposição:

- Conta Azure já logada
- Cloud Shell funcionando
- Uma boa xícara de café ☕

Avaliação dos arquivos Dockerfile e Python

- Localize o diretório container usando o VS Code;
- Explore o arquivo Dockerfile, veja seu conteúdo;
- Explore o arquivo app.py, veja seu conteúdo;
- Identifique as funções
 - listar_produtos
 - carregar_produtos
 - health
- Explore os demais arquivos do diretório



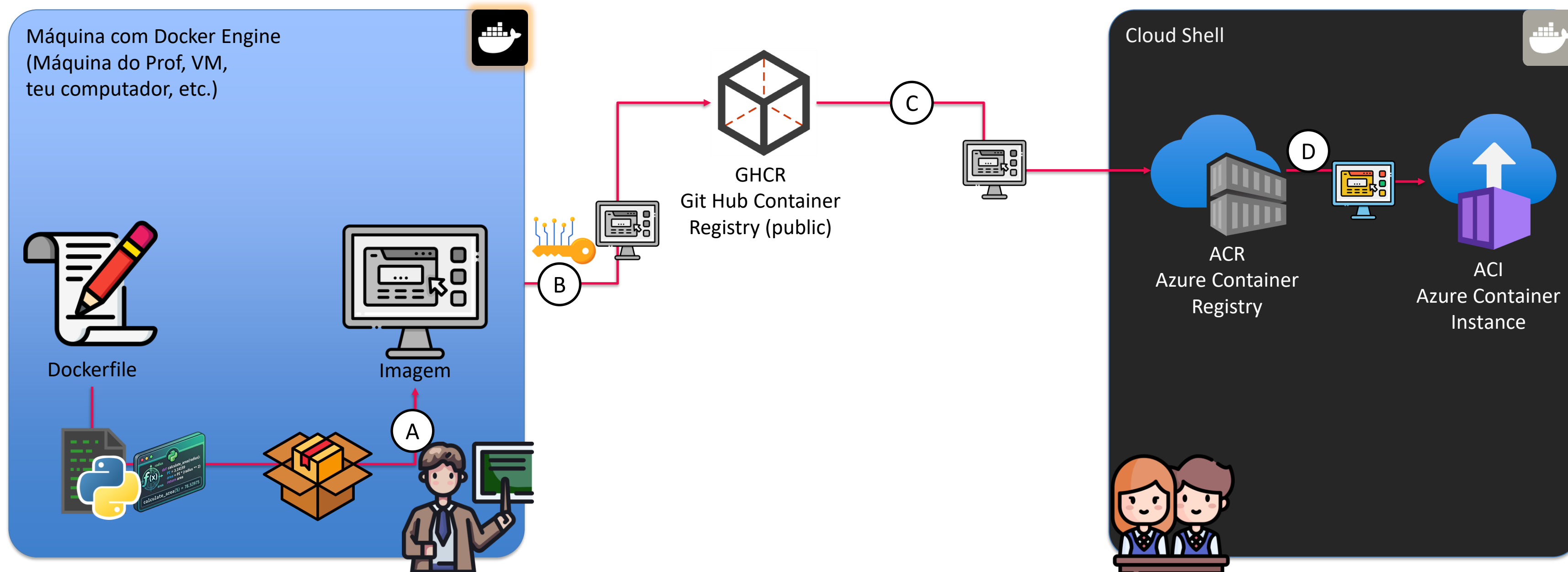
```
import json
import logging
import azure.functions as func

app = func.FunctionApp(http_auth_level=func.AuthLevel.ANONYMOUS)

# Mock inicial — depois substituído pelo Blob na v2-blob/
PRODUTOS MOCK = [
    {"id": 1, "nome": "Cadeira Ergonômica DXRacer",
     "categoria": "moveis", "preco": 1499.90},
    ...
]

@app.route(route="produtos", methods=["GET"])
def listar_produtos(req: func.HttpRequest) -> func.HttpResponse:
    # Início da função
    ...
    # Retorno da função
    return func.HttpResponse(
        json.dumps({"total": len(resultado),
                    "produtos": resultado}, ensure_ascii=False),
        mimetype="application/json",
        status_code=200,
    )

@app.route(route="health", methods=["GET"])
def health(req: func.HttpRequest) -> func.HttpResponse:
    return func.HttpResponse(
        json.dumps({"status": "ok", "service": "qc-catalogo",
                    "source": "mock"}),
        mimetype="application/json",
    )
```



A) A imagem é gerada em uma máquina que possua o Docker Engine

Nota: Nas contas Azure for Students, o ACR Tasks é bloqueado (az acr build → TasksOperationsNotAllowed) e o Cloud Shell não tem daemon Docker (docker build não roda). Por isso a imagem é construída e publicada uma vez no GHCR pelo professor, e cada aluno só importa a imagem pronta para o seu ACR via az acr import (operação permitida, sem Tasks e sem Docker local).

B) Então, a imagem é transferida para um Container Registry público (Usei o Git Hub CR)

Obs.: Necessário PAT (Personal Access Token)

C) A imagem então pode ser compartilhada e copiada para um CR privado. Neste caso, usamos o ACR

D) Por último, a instância será criada a partir do Terraform

Cópia da imagem e criação da instância

Identifica o nome do ACR

```
ACR_NAME=$(cd ~/aie-cloud/aulas/03-serverless-containers/lab/terraform && terraform output -raw acr_name)
```

Importa a imagem

```
az acr import \  
  --name "$ACR_NAME" \  
  --source ghcr.io/isaiasbrito/produtos-api:v1 \  
  --image produtos-api:v1 \  
  --force
```

```
az acr repository list -n "$ACR_NAME" -o table
```

Depois da importação, habilitar o ACI

```
cd ~/aie-cloud/aulas/03-serverless-containers/lab/terraform  
terraform apply -auto-approve -var="aci_enabled=true"
```

Testar a ACI (obtem e exibe URL)

```
ACI_FQDN=$(cd ~/aie-cloud/aulas/03-serverless-containers/lab/terraform && terraform output -raw aci_fqdn)
```

```
sleep 60 # aguardar a instância subir e exibir URL
```

```
echo "http://$ACI_FQDN:8080/health"
```

```
echo "http://$ACI_FQDN:8080/produtos?categoria=moveis"
```



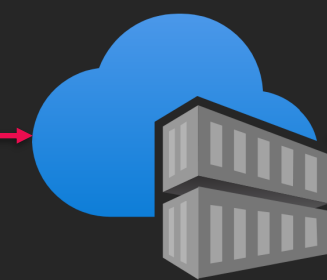
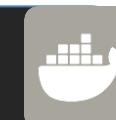
C



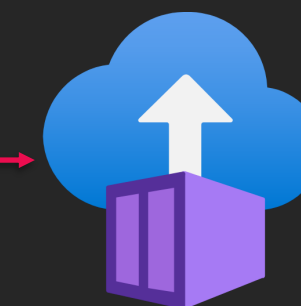
D



Cloud Shell



ACR
Azure Container
Registry



ACI
Azure Container
Instance



EXERCÍCIO

Exercício Aula 3

Exercício que vale 10% da nota

Esta é a segunda entrega de grupo da disciplina. Ao todo são 5 entregas intermediárias (10% cada) + projeto integrado final entregue como ZIP 1 semana após a Aula 6 (50%, sem apresentação oral).

Como o grupo se organiza

Os 3 níveis de exercícios são divisão de trabalho dentro do grupo, não escolha individual livre:

🟢 *Nível 1 — Básico: serverless, Managed Identity, Function vs Container, Dockerfile review*

🟡 *Nível 2 — Intermediário: segunda tool (cálculo de frete), Application Insights e observabilidade, endurecimento/dimensionamento do ACI*

🔴 *Nível 3 — Avançado: bônus opcional — spec de tool para agente AI, benchmark de carga, CI/CD com GitHub Actions + OIDC*

Mínimo obrigatório: N1 + N2 cobertos. N3 é bônus (até +2 pts extras na nota da aula).

Repo Base: <https://github.com/elthonf/aie-cloud.git>

Item	Obrigatório?	Pontos máximos
Cabeçalho do grupo + distribuição do trabalho	✔ Sim	1 pt (Critério 4)
🟢 N1 — Exercícios 1.1 (quando usar serverless), 1.2 (MI vs alternativas), 1.3 (cold start), 1.4 (Dockerfile review)	✔ Sim	3 pts (Critério 1)
🟡 N2 — 2.1 (segunda tool: cálculo de frete), 2.2 (App Insights), 2.3 (ACI: restart/sizing/secure env)	✔ Sim	3 pts (Critério 2) + 2 pts qualidade técnica (Critério 3)
🔴 N3 — 3.1 (spec de tool de agente), 3.2 (benchmark), 3.3 (CI/CD GitHub Actions + OIDC)	🎁 Bônus	até +2 pts extras
Reflexão coletiva ao final	✔ Sim	1 pt (Critério 5)
Total		10 pts

Sugestão de nome de arquivo: `entrega-grupo-NN-aula03.zip` (substitua NN pelo nome de um integrante).

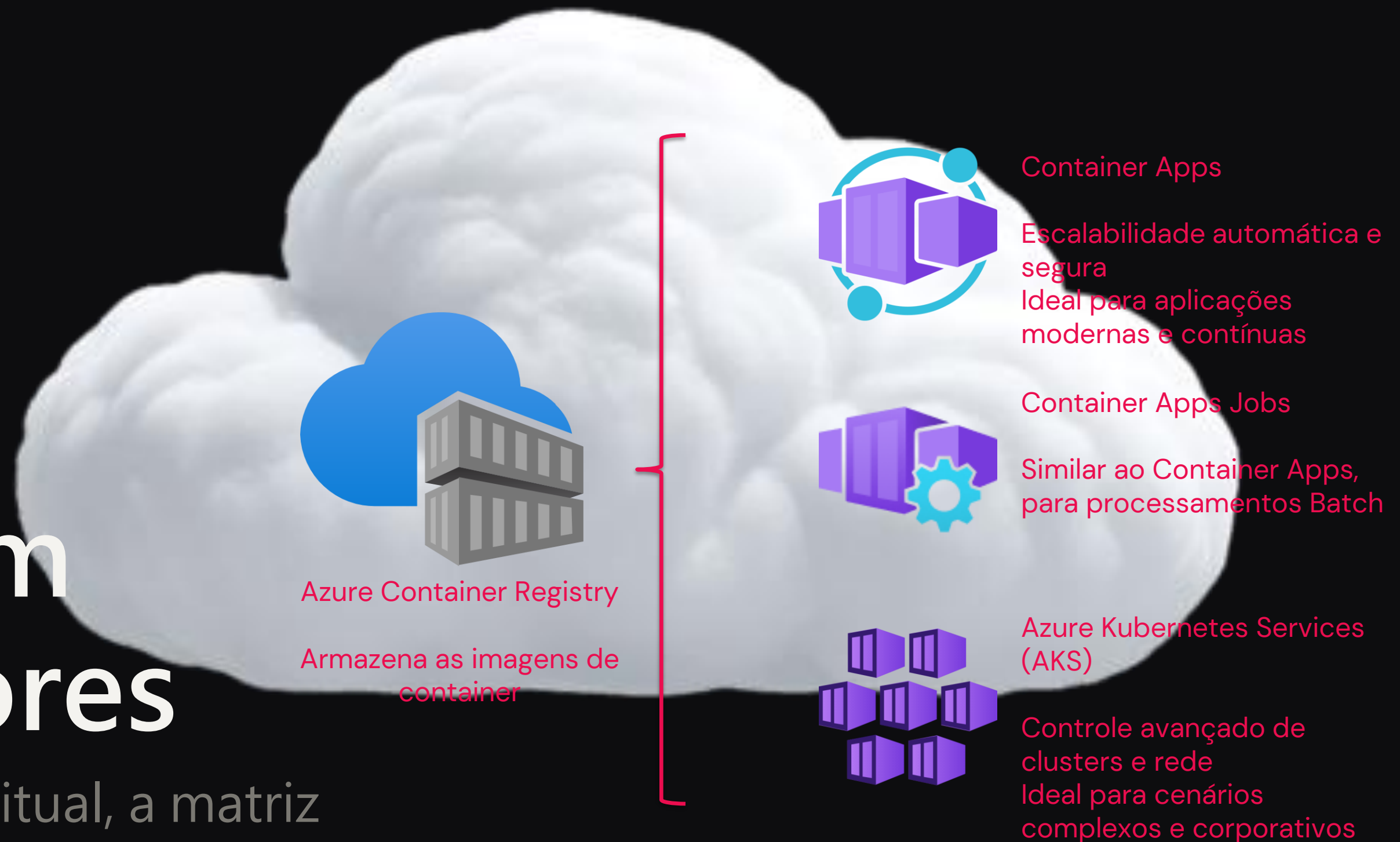
O que deve conter no arquivo

```
qc-grupo-NN-aula03/
├─ entrega-grupo-aula03.md      # ★ documento principal (template em ../template-entrega-grupo.md)
├─ terraform/                  # main.tf evoluído (Function + ACR + ACI + App Insights se N2.2)
├─ function/                   # Código Python da Function (incluindo segunda tool se N2.1)
├─ docker/                     # Dockerfile + app.py FastAPI (se entregue)
├─ .github/workflows/          # workflow CI/CD (se N3.3)
├─ diagramas/
│   └─ arquitetura-qc-aula03.png # Diagrama da QC com a camada de API/compute
└─ README.md                   # Como rodar o N2/N3 (se incluído)
```

04

Container em Outros Sabores

Container Apps, AKS conceitual, a matriz
Function/ACI/Apps/AKS, o destroy obrigatório e a
API como tool dos agentes.



O meio-termo que faltava

"**Serverless para containers**" — lançado em 2022, padrão emergente. Auto-scale de **0 a N** réplicas via KEDA (Kubernetes Event-driven Autoscaling): escala a zero como Function, mas roda qualquer container.

Ingress HTTPS gerenciado + certificado automático + revisões blue/green embutidas — o que o ACI (Azure Container Instance) não dá.

SCALE TO ZERO

Custo idle zerado — paga por vCPU-segundo quando ativo.

FRAMEWORK PRÓPRIO

FastAPI, Spring, Django — o que não cabe no modelo de Function.

QC: WORKER DE PEDIDOS

Processamento assíncrono com tráfego variável vira Container Apps.

Function / ACI / Container Apps / AKS

CENÁRIO	RECOMENDADO	POR QUÊ
API simples, <1min, orientada a evento	Function	Pay-per-call, deploy rápido
Tarefa one-shot, demo, dev/test	ACI	O mais simples, pay-per-second
Microserviço com framework próprio	Container Apps	Auto-scale, ingress, revisões
Plataforma 20+ serviços, service mesh	AKS	Controle fino, ecossistema K8s
Migração de monolito legado (Java/PHP)	App Service	PaaS clássico, runtime gerenciado

catálogo de hoje começa Function · para executar uma instância de container, usamos o ACI
 worker maior vira Container Apps · plataforma de agentes em escala, no futuro, é AKS.

Destrua tudo. **Confirme o zero.**

✗ NÃO BASTA

Parar os recursos

Container parado, ACR ocioso — ainda há storage e registro reservados consumindo. Pausar não é zerar.

✓ FAÇA

terraform destroy

Apaga RG, Function, Service Plan, ACR e ACI de uma vez. Depois: Cost Management → confirme ~\$0.

```
terraform destroy -auto-approve
```

A URL da Function é uma tool de agente

O endpoint `/api/produtos` é funciona como **catálogo de um agente**.

Os agentes o chamam como **tool** — exatamente este JSON vira a definição de uma ferramenta no formato function calling.

- **Function** — vai pra produção (TLS, scale, pay-per-call)
- **Container no ACR** — deploy alternativo, pronto pra ACI/Apps

```
// Tool definition para um agente {  
  
"name": "buscar_produtos" ,  
  
"description": "Busca produtos da QC          por categoria ou nome" ,  
  
"url": "https://<func>.azurewebsites.net/api/produtos" ,  
  
"parameters" : {  
  
"categoria": "string" ,  
  
"nome": "string"  }  
}
```

O que você leva desta aula

SERVERLESS

Event-driven + pay-per-call

A função roda sob demanda e custa zero ociosa. O free tier cobre a maioria das tools.

TRIGGERS + MANAGED IDENTITY

HTTP dispara, MI protege

O agente chama via HTTP. E a credencial nunca toca o código — a vacina da aula.

CONTAINERS

Docker empacota, ACl roda

Mesmo código, deploy diferente. Imagem leve, registry, container no segundo.

MATRIZ + CUSTO

Decisão = custo & escala

Function / ACl / Apps / AKS conforme o cenário. E destrói tudo no final.



Obrigado.

Nos vemos na Aula 4